

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**



**PBL1. ĐỒ ÁN
LẬP TRÌNH TÍNH TOÁN**

**ĐỀ TÀI: BÀI TOÁN TÌM ĐƯỜNG ĐI NGẮN
NHẤT BẰNG THUẬT TOÁN DIJKSTRA, FLOYD**

GIẢNG VIÊN HƯỚNG DẪN: Đỗ Thị Tuyết Hoa

SINH VIÊN THỰC HIỆN:

Nguyễn Công Dũng MSSV: 102250113 LỚP: 25T_DT3

Nguyễn Văn Tài MSSV: 102250143 LỚP: 25T_DT3

Đà Nẵng, tháng 06/2026

LỜI MỞ ĐẦU

Trong kỷ nguyên công nghệ số hiện nay, sự phát triển mạnh mẽ của các hệ thống hạ tầng thông minh như mạng máy tính, logistics và giao thông đô thị đã đặt ra những thách thức lớn về bài toán tối ưu hóa thời gian cũng như chi phí. Trong đó, tìm đường đi ngắn nhất trên đồ thị là một bài toán cổ điển nhưng sở hữu giá trị thực tiễn vô cùng to lớn. Đây chính là lõi công nghệ vận hành cho các hệ thống định vị bản đồ trực tuyến, tối ưu hóa tuyến đường vận tải hay định tuyến các gói dữ liệu trong không gian mạng.

Nhận thức được tầm quan trọng đó, trong học phần **PBL1: Đồ án lập trình tính toán**, nhóm em đã lựa chọn triển khai đề tài: "**Bài toán tìm đường đi ngắn nhất bằng thuật toán Dijkstra, Floyd**".

Báo cáo này là thành quả của quá trình nhóm nghiên cứu từ lý thuyết toán học đến việc hiện thực hóa mã nguồn. Đề tài tập trung vào việc phân tích cơ chế hoạt động, đánh giá ưu - nhược điểm của hai giải thuật kinh điển là Dijkstra và Floyd-Warshall. Từ đó, nhóm tiến hành cài đặt chương trình bằng ngôn ngữ lập trình nhằm tính toán, đưa ra các kết quả đối sánh hiệu năng và xây dựng không gian mô phỏng trực quan cho bài toán.

Mặc dù đã có nhiều cố gắng trong quá trình tìm hiểu và phối hợp làm việc, tuy nhiên do kiến thức chuyên môn và kinh nghiệm thực tế của các sinh viên năm thứ nhất còn hạn chế, đồ án chắc chắn không tránh khỏi những thiếu sót. Nhóm em rất mong nhận được những ý kiến đóng góp, phê bình quý báu từ các thầy cô giáo trong bộ môn để đề tài được hoàn thiện và phát triển tốt hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI MỞ ĐẦU.....	2
MỤC LỤC	3
DANH MỤC HÌNH VẼ	5
1. GIỚI THIỆU ĐỀ TÀI.....	6
1.1. Lý do lựa chọn đề tài	6
1.2. Mục tiêu của đề tài.....	6
1.3. Phạm vi nghiên cứu	7
1.4. Phương pháp nghiên cứu	7
2. CƠ SỞ LÝ THUYẾT	7
2.1. Ý tưởng	7
2.1.1. Ý tưởng thuật toán Dijkstra.....	8
2.1.2. Ý tưởng thuật toán Floyd-Warshall.....	8
2.2. Cơ sở lý thuyết.....	8
2.2.1. Một số định nghĩa cơ bản	8
2.2.2. Cơ sở lý thuyết thuật toán Dijkstra.....	9
2.2.3. Cơ sở lý thuyết thuật toán Floyd-Warshall	10
3. CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	11
3.1. Phát biểu bài toán.....	11
3.1.1. Dữ liệu đầu vào (Input)	11
3.1.2. Kết quả đầu ra (Output).....	11
3.2. Cấu trúc dữ liệu.....	12
3.2.1. Cấu trúc dữ liệu lưu trữ đồ thị.....	12
3.2.2. Cấu trúc dữ liệu hàng đợi ưu tiên - Heap	15
3.2.3. Cấu trúc dữ liệu lưu trữ kết quả tối ưu và truy vết lộ trình	17
3.2.4. Cấu trúc dữ liệu quản lý các đối tượng đồ họa.....	18
3.3. Thuật toán	19
3.3.1. Cài đặt thuật toán Dijkstra.....	19
3.3.2. Cài đặt thuật toán Floyd	22

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ	24
4.1. Tổ chức chương trình.....	24
4.1.1. Tổ chức tệp tin.....	24
4.1.2. Tổ chức luồng thực thi chương trình.....	25
4.2. Ngôn ngữ cài đặt.....	27
4.3. Kết quả.....	27
4.3.1. Giao diện chính của chương trình	27
4.3.2. Kết quả thực thi của chương trình	29
4.3.3. Nhận xét.....	32
5. KẾT LUẬN	33
5.1. Kết quả đạt được	33
5.2. Hạn chế	33
5.3. Hướng phát triển	34
TÀI LIỆU THAM KHẢO	35
PHỤ LỤC	36
header.h.....	36
algo.h.....	38
physics.h	42
graphics.h.....	46
main.cpp.....	61

DANH MỤC HÌNH VẼ

Hình 1: Giao diện trực quan hóa đồ thị và tìm đường đi ngắn nhất của ứng dụng	6
Hình 2: Màn hình hiển thị kết quả đường đi ngắn nhất.....	11
Hình 3: Đồ thị vô hướng có trọng số	13
Hình 4: Đồ thị có hướng có trọng số	13
Hình 5: Cấu trúc dữ liệu danh sách kề	14
Hình 6: Cấu trúc dữ liệu hàng đợi ưu tiên heap	15
Hình 7: Cấu trúc dữ liệu đồ họa dành cho đỉnh.....	18
Hình 8: Cấu trúc dữ liệu đồ họa dành cho cạnh	19
Hình 9: Cài đặt hàm xử lý thuật toán Dijkstra.....	21
Hình 10: Cài đặt hàm truy vết đường đi ngắn nhất đối với thuật toán Dijkstra	22
Hình 11: Cài đặt hàm xử lý thuật toán Floyd	23
Hình 12: Cài đặt hàm truy vết đường đi ngắn nhất đối với thuật toán Floyd.....	23
Hình 13: Sơ đồ tổ chức luồng thực thi tổng thể chương trình.....	26
Hình 14: Giao diện khu vực quản lý dữ liệu đầu vào của đồ thị.....	28
Hình 15: Giao diện tổng quan của chương trình khi vừa khởi chạy	29
Hình 16: Kết quả đường đi ngắn nhất theo thuật toán Dijkstra.....	30
Hình 17: Kết quả đường đi ngắn nhất theo thuật toán Floyd	30
Hình 18: Báo lỗi thuật toán Dijkstra khi đồ thị tồn tại cạnh âm.....	31
Hình 19: Báo lỗi thuật toán Floyd khi đồ thị tồn tại chu trình âm	31
Hình 20: Chương trình không tìm thấy đường đi giữa hai đỉnh trên hai thành phần liên thông khác nhau của đồ thị	32

1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do lựa chọn đề tài

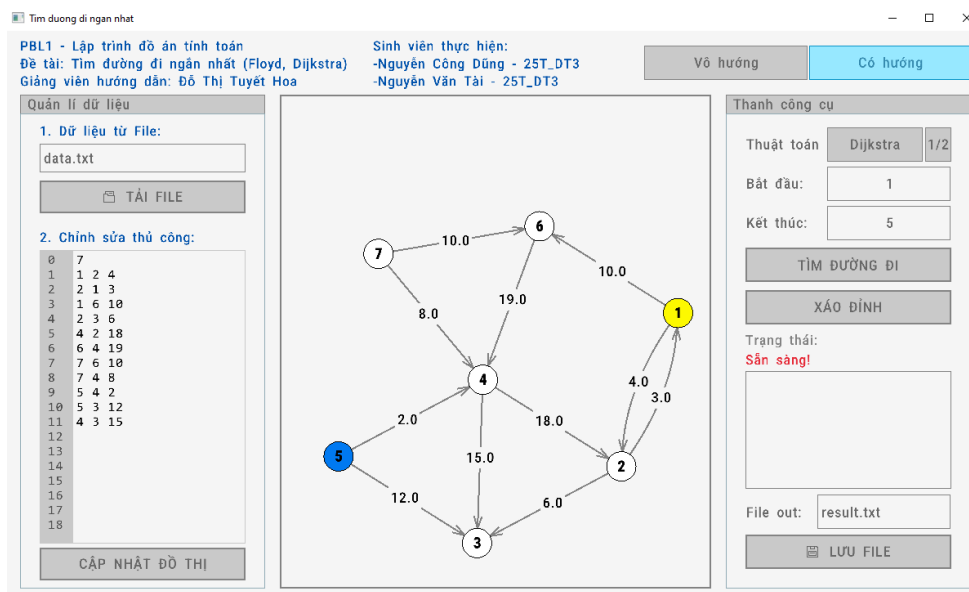
Mục đích của đề tài là nghiên cứu cơ sở lý thuyết của bài toán tìm đường đi ngắn nhất trên đồ thị, đồng thời xây dựng một chương trình có khả năng áp dụng chính xác và hiệu quả hai giải thuật Dijkstra và Floyd-Warshall. Thông qua việc thực hiện đề tài, nhóm sinh viên có cơ hội vận dụng kiến thức về lý thuyết đồ thị, cấu trúc dữ liệu và lập trình, từ đó nâng cao kỹ năng đối sánh hiệu năng thuật toán cũng như giải quyết các bài toán tối ưu hóa trong thực tế.

1.2. Mục tiêu của đề tài

Về mặt lý thuyết: Nghiên cứu sâu và nắm vững bản chất toán học, cơ chế hoạt động, độ phức tạp thuật toán và phạm vi áp dụng của hai giải thuật tìm đường đi ngắn nhất kinh điển: thuật toán Dijkstra và thuật toán Floyd-Warshall.

Về mặt cài đặt chương trình: Sử dụng ngôn ngữ C++ phối hợp với cấu trúc dữ liệu phù hợp để cài đặt chính xác cả hai thuật toán. Xây dựng chương trình có khả năng xử lý, tính toán dữ liệu đồ thị linh hoạt thông qua việc đọc/ghi file cấu trúc và tương tác trực tiếp trên giao diện đồ họa.

Về mặt trực quan hóa: Ứng dụng thư viện đồ họa để mô phỏng trực quan cấu trúc đồ thị, và hiển thị đường đi ngắn nhất. Đồng thời, tích hợp giải thuật dàn trải đồ thị tự động giúp tối ưu hóa không gian hiển thị nút và cạnh, hỗ trợ người dùng quan sát cấu trúc dữ liệu một cách rõ ràng nhất.



Hình 1: Giao diện trực quan hóa đồ thị và tìm đường đi ngắn nhất của ứng dụng

1.3. Phạm vi nghiên cứu

- Về cấu trúc dữ liệu đồ thị: Đề tài tập trung nghiên cứu trên cấu trúc đồ thị hữu hạn (có hướng hoặc vô hướng) có trọng số.
- Về tập dữ liệu thực nghiệm: Chương trình tập trung xử lý các bộ dữ liệu đồ thị dạng vừa và nhỏ (dưới vài trăm đỉnh) để đảm bảo tính trực quan khi mô phỏng trên không gian giao diện phần mềm.
- Về môi trường cài đặt: Đề tài giới hạn trong việc nghiên cứu và xây dựng ứng dụng chạy trên môi trường hệ điều hành máy tính cá nhân, sử dụng ngôn ngữ lập trình C++ và thư viện Raylib để hỗ trợ hiển thị.

1.4. Phương pháp nghiên cứu

- Phương pháp nghiên cứu lý thuyết:

Thu thập, đọc hiểu và phân tích các tài liệu khoa học, giáo trình cấu trúc dữ liệu và giải thuật chuyên ngành nhằm nắm vững nền tảng toán học của lý thuyết đồ thị. Nghiên cứu sâu về cơ chế vận hành, điều kiện biên, cách tối ưu hóa và chứng minh tính đúng đắn của hai thuật toán: Dijkstra và Floyd-Warshall.

- Phương pháp mô hình hóa và thực nghiệm:

Chuyển đổi các bài toán mạng lưới thực tế thành mô hình đồ thị học toán học $G = (V, E)$ được cấu trúc hóa trong máy tính. Sử dụng ngôn ngữ lập trình C++ để thiết kế cấu trúc dữ liệu lưu trữ và cài đặt lõi thuật toán. Đồng thời ứng dụng thư viện đồ họa Raylib để xây dựng ứng dụng mô phỏng trực quan, thiết kế giao diện tương tác người dùng và xử lý đọc/ghi file dữ liệu cấu trúc.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

Để giải quyết bài toán tìm đường đi ngắn nhất, trước hết ta cần mô hình hóa mạng lưới giao thông hoặc sơ đồ thực tế thành một đồ thị toán học $G = (V, E)$. Trong đó:

- Tập hợp các đỉnh (V): Đại diện cho các địa điểm hoặc nút giao thông.

- Tập hợp các cạnh (E): Đại diện cho các con đường kết nối giữa các địa điểm. Mỗi cạnh sẽ đi kèm với một trọng số tương ứng với khoảng cách hoặc chi phí di chuyển.

Hai thuật toán tìm đường đi ngắn nhất có hai cách tiếp cận khác nhau để tối ưu hoá bài toán, cụ thể:

2.1.1. Ý tưởng thuật toán Dijkstra

Mục đích của thuật toán: Giải quyết bài toán tìm đường đi ngắn nhất từ một đỉnh nguồn cố định đơn nhất đến tất cả các đỉnh còn lại trên đồ thị.

Ý tưởng: Vận dụng chiến lược tham lam, thuật toán luôn chọn ra đỉnh có chi phí tạm thời nhỏ nhất trong số các đỉnh chưa được tối ưu để cố định chi phí. Sau đó, sử dụng đỉnh này làm bước đệm để cập nhật lại chi phí tối ưu hơn cho các đỉnh kề xung quanh nó.

2.1.2. Ý tưởng thuật toán Floyd-Warshall

Mục đích của thuật toán: Giải quyết bài toán tìm đường đi ngắn nhất giữa mọi cặp đỉnh trên đồ thị.

Ý tưởng: Vận dụng chiến lược quy hoạch động, thuật toán tiến hành tối ưu hóa đường đi từ đỉnh i đến đỉnh j bằng cách so sánh chi phí hiện tại với chi phí của lộ trình đi vòng qua đỉnh trung gian k ($i, j, k = \overline{1, V}$).

2.2. Cơ sở lý thuyết

2.2.1. Một số định nghĩa cơ bản

- Đồ thị vô hướng gồm một tập hợp các đỉnh và một tập hợp các cạnh kết nối các cặp đỉnh đó, trong đó các cạnh không quy định chiều hay hướng di chuyển. Nói cách khác, nếu tồn tại một cạnh nối giữa hai đỉnh A và B, mối quan hệ này mang tính hai chiều, cho phép di chuyển tự do từ A sang B và ngược lại từ B về A.

- Đồ thị có hướng gồm một tập hợp các đỉnh và một tập hợp các cạnh kết nối các cặp đỉnh đó, trong đó các cạnh được quy định chiều di chuyển rõ ràng, gọi là các cung. Nếu có một cung đi từ đỉnh A sang đỉnh B, điều này chỉ xác định đường đi một chiều từ A đến B.

- Đồ thị có trọng số là đồ thị (có thể là vô hướng hoặc có hướng) mà mỗi cạnh hoặc cung trên đó được gán kèm một giá trị số thực gọi là trọng số. Trọng số này đại diện cho một đại lượng đo lường thực tế giữa hai nút giao thông, tùy

thuộc vào ngữ cảnh của bài toán như khoảng cách địa lý, chi phí tài chính, thời gian di chuyển hoặc độ trễ tín hiệu mạng.

- Đường đi giữa hai đỉnh trên đồ thị là một dãy luân phiên các đỉnh và các cạnh kết nối chúng, bắt đầu từ đỉnh nguồn và kết thúc ở đỉnh đích. Một đường đi được gọi là đường đi đơn nếu nó không đi qua bất kỳ đỉnh nào quá một lần.
- Độ dài đường đi trên đồ thị có trọng số là đại lượng được xác định bằng tổng tất cả các trọng số của các cạnh cấu thành nên đường đi đó.
- Đường đi ngắn nhất giữa hai đỉnh trên đồ thị có trọng số là đường đi hợp lệ kết nối giữa hai đỉnh đó sao cho tổng chi phí của đường đi đạt giá trị nhỏ nhất (tối ưu toàn cục).

2.2.2. Cơ sở lý thuyết thuật toán Dijkstra

Thuật toán Dijkstra được xây dựng dựa trên các cơ sở lý thuyết sau đây:

- Thuật toán tham lam (greedy algorithm) là chiến lược luôn lập tức lựa chọn phương án tốt nhất ở thời điểm hiện tại mà không tính toán đến các hệ quả tương lai. Vận dụng tư duy này, thuật toán Dijkstra tại mỗi bước lặp luôn chọn ngay đỉnh có khoảng cách tạm thời nhỏ nhất để cố định nhãn đường đi ngắn nhất.
- Giãn cạnh: Sau khi dán nhãn cố định cho một đỉnh, thuật toán Dijkstra sử dụng đỉnh vừa chọn làm bước đệm để kiểm tra xem liệu lộ trình đi từ đỉnh nguồn qua đỉnh đệm này đến các đỉnh kề xung quanh có chi phí thấp hơn khoảng cách hiện tại hay không. Nếu có, nhãn khoảng cách và đỉnh lưu vết của các đỉnh kề sẽ lập tức được cập nhật lại theo giá trị tối ưu hơn.
- Cấu trúc Min-Heap là cấu trúc dữ liệu giúp tối ưu hóa hiệu năng bằng cách luôn giữ đỉnh có khoảng cách nhỏ nhất ở gốc cây. Thay vì dùng mảng thường tốn thời gian $O(V)$ để tìm kiếm, Min-Heap giúp lập tức trích xuất đỉnh tối ưu này và cập nhật khoảng cách cho các đỉnh kề chỉ với độ phức tạp $O(\log V)$. Nhờ đó, tổng thời gian chạy của thuật toán giảm đáng kể, giúp ứng dụng xử lý mượt mà các đồ thị quy mô lớn.

Điều kiện áp dụng: Thuật toán Dijkstra bắt buộc phải hoạt động trên đồ thị có trọng số không âm. Nếu đồ thị chứa cạnh có trọng số âm, chiến lược tham lam của giải thuật sẽ không còn chính xác.

Với điều kiện trên, tính đúng đắn của thuật toán được chứng minh như sau:

- Bài toán: Cho đồ thị $G = (E, V)$ có trọng số. Tìm đường đi ngắn nhất giữa đỉnh s và các đỉnh còn lại.
- Gọi $d[i]$ là khoảng cách tạm thời từ đỉnh s đến đỉnh i . Trọng số của cạnh (i, j) được kí hiệu $w(i, j)$. $(i, j = \overline{1, V})$
- Với cách tiếp cận tham lam, tại một bước của thuật toán, ta có u là đỉnh có $d[u]$ nhỏ nhất chưa được dán nhãn.
- Giả sử $d[u]$ không phải là chi phí tối ưu của đường đi từ s đến u . Suy ra tồn tại một lộ trình P ngắn hơn.
- Giả sử lộ trình P này đi qua đỉnh v gần u nhất, chưa được dán nhãn, ta có:

$$d[v] + w(v, u) \leq d[u] \quad , \quad w(v, u) > 0$$

$\Rightarrow d[v] < d[u]$: Điều này vi phạm điều kiện ban đầu vì $d[u]$ đã là nhỏ nhất

$\Rightarrow \nexists v$ thoả mãn trường hợp trên.

- Do đó, $d[u]$ thoả mãn là chi phí tối ưu nhất của đường đi từ s đến u .

2.2.3. Cơ sở lý thuyết thuật toán Floyd-Warshall

Thuật toán Floyd-Warshall được xây dựng dựa trên các cơ sở lý thuyết sau:

- Quy hoạch động (dynamic programming) là cách giải bài toán lớn dựa trên kết quả của các bài toán con tương tự. Thay vì tìm đường đi trực tiếp giữa các cặp đỉnh một cách độc lập, Floyd-Warshall giải quyết bài toán bằng cách chia nhỏ nó thành các bài toán con dựa trên việc kiểm tra các đỉnh trung gian. Kết quả tối ưu của bước sau sẽ được xây dựng và kế thừa trực tiếp từ kết quả tối ưu của các bước trước đó, giúp tránh được việc phải tính toán lại từ đầu.

- Kỹ thuật tối ưu: Khi thuật toán xét đến một đỉnh trung gian k bất kỳ, nó sẽ lựa chọn trường hợp tối ưu hơn: giữ nguyên lộ trình cũ không qua k , hoặc đi vòng từ i sang k rồi từ k mới sang j .

Điều kiện áp dụng: Thuật toán Floyd-Warshall có thể hoạt động trên cả đồ thị có trọng số dương và trọng số âm. Tuy nhiên, điều kiện bắt buộc là đồ thị không được xuất hiện chu trình âm, vì chu trình âm sẽ làm cho độ dài đường đi giảm xuống vô hạn, khiến bài toán tối ưu không thể cho ra kết quả.

3. CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

Cho một đồ thị gồm các đỉnh và các cạnh có hướng hoặc vô hướng kết nối giữa chúng (sau đây gọi chung là cạnh). Mỗi cạnh được gán một trọng số. Mục tiêu của bài toán là tìm ra một lộ trình từ một đỉnh xuất phát đến một đỉnh đích sao cho tổng chi phí lộ trình đó là nhỏ nhất.

Để hiện thực hóa các mô hình toán học thành một ứng dụng phần mềm chạy được, bài toán được cấu trúc cụ thể thông qua các thành phần dữ liệu sau:

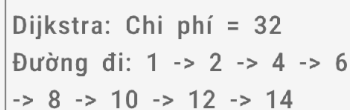
3.1.1. Dữ liệu đầu vào (Input)

- Loại đồ thị: là đồ thị có hướng hoặc vô hướng.
- Số lượng đỉnh n (n là số nguyên dương và n nhỏ hơn hoặc bằng 100).
- Tập hợp các cạnh nối được biểu diễn dưới dạng danh sách các bộ ba số $x\ y\ z$ ($x, y \in \mathbb{N}$, $1 \leq x, y \leq n$, $z \in \mathbb{R}$ và nằm trong giới hạn kiểu dữ liệu số thực float). Nếu đồ thị có hướng, (x, y, z) được hiểu là cung $x \rightarrow y$ có trọng số z . Nếu đồ thị vô hướng, bộ ba số trên được hiểu là cạnh $x-y$ có trọng số z .
- Một đỉnh s là đỉnh bắt đầu, và một đỉnh t là đỉnh kết thúc.

Trong chương trình, dữ liệu trên được ghi vào thông qua hai phương thức: đọc từ file (người dùng phải nhập tên file) hoặc do người dùng trực tiếp chỉnh sửa thủ công bằng bàn phím để tạo đỉnh mới hoặc cạnh mới và điều chỉnh trọng số của các cạnh hoặc cung.

3.1.2. Kết quả đầu ra (Output)

- Tổng độ dài (chi phí) của đường đi ngắn nhất từ đỉnh bắt đầu s đến đỉnh đích t , là một số thực duy nhất có độ chính xác đến 02 chữ số thập phân.
- Danh sách chuỗi các đỉnh đi qua theo đúng thứ tự hành trình, là đường đi ngắn nhất từ s đến t .



```
Dijkstra: Chi phí = 32
Đường đi: 1 -> 2 -> 4 -> 6
-> 8 -> 10 -> 12 -> 14
```

Hình 2: Màn hình hiển thị kết quả đường đi ngắn nhất

3.2. Cấu trúc dữ liệu

Đầu tiên, chương trình định nghĩa hai hằng số sau để thiết lập giới hạn cho dữ liệu:

- `#define N 105`: là số lượng đỉnh tối đa cấu thành nên đồ thị mà chương trình có thể quản lý, được khai báo dư nhằm tránh lỗi tràn chỉ số mảng.
- `#define inf 1e30`: là giá trị vô cùng lớn (10^{30}), là giới hạn trên của mọi trọng số của mọi cạnh đồ thị.

3.2.1. Cấu trúc dữ liệu lưu trữ đồ thị

3.2.1.1. Ma trận kề

Cấu trúc được dùng để lưu ma trận kề là hệ thống sử dụng mảng hai chiều toàn cục để lưu trữ dữ liệu của đồ thị:

```
float a[N][N]
```

- Ô $a[i][j]$ ($i, j \in \{1, 2 \dots n\}$) lưu trữ trọng số của cạnh đi từ đỉnh i đến đỉnh j . Trong đồ thị vô hướng, ta luôn có: $a[i][j] = a[j][i]$.
- Ô $a[i][i]$ mặc định bằng 0 (muốn đi từ một đỉnh i đến chính nó, chỉ cần đứng yên – tốn 0 chi phí).
- Nếu giữa hai đỉnh không có cạnh nối trực tiếp, $a[i][j]$ sẽ được gán giá trị mặc định là vô cùng lớn (inf).

Giới hạn số hàng và số cột tối đa của mảng hai chiều a là hằng số N .

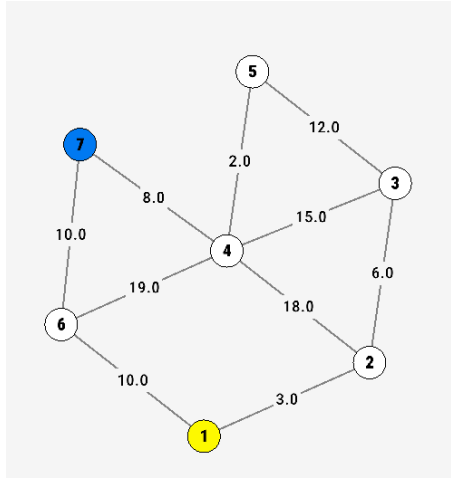
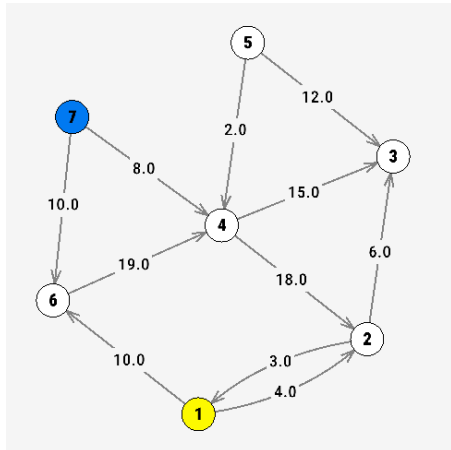
Trong đồ án này, nhóm em quyết định lựa chọn ma trận kề làm cấu trúc dữ liệu lưu trữ đồ thị cho thuật toán Floyd-Warshall. Floyd-Warshall là thuật toán tìm đường đi ngắn nhất giữa mọi cặp đỉnh, hoạt động dựa trên cấu trúc 3 vòng lặp lồng nhau với công thức truy hồi: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$. Việc sử dụng Ma trận kề cho phép máy tính truy xuất ngay lập tức giá trị tại tọa độ dòng i , cột j với độ phức tạp thời gian lý tưởng là $O(1)$.

Từ dữ liệu đầu vào là danh sách các bộ ba số $x \ y \ z$, quy trình chuyển đổi và cấu trúc hóa dữ liệu thành ma trận kề được thực hiện tuần tự qua các bước sau:

- 1) Ban đầu, khởi tạo mảng hai chiều $a[i][j] = \begin{cases} 0 & \text{nếu } i = j \\ \text{inf} & \text{nếu } i \neq j \end{cases}$

- 2) Đọc dữ liệu đầu vào, với mỗi $x \ y \ z$, ta cập nhật lại $a[x][y] = z$ cho đồ thị có hướng, và $a[x][y] = a[y][x] = z$ cho đồ thị vô hướng.

Để minh họa trực quan và làm rõ cách thức số hóa cấu trúc của đồ thị vào bộ nhớ máy tính thông qua ma trận kề, ta xét hai mô hình đồ thị mẫu:

Đồ thị	Ma trận kề																																																																
	<table><tr><th>a</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th><th>7</th></tr><tr><th>1</th><td>0</td><td>3</td><td>∞</td><td>∞</td><td>∞</td><td>10</td><td>∞</td></tr><tr><th>2</th><td>3</td><td>0</td><td>6</td><td>18</td><td>∞</td><td>∞</td><td>∞</td></tr><tr><th>3</th><td>∞</td><td>6</td><td>0</td><td>15</td><td>12</td><td>∞</td><td>∞</td></tr><tr><th>4</th><td>∞</td><td>18</td><td>15</td><td>0</td><td>2</td><td>19</td><td>8</td></tr><tr><th>5</th><td>∞</td><td>∞</td><td>12</td><td>2</td><td>0</td><td>∞</td><td>∞</td></tr><tr><th>6</th><td>10</td><td>∞</td><td>∞</td><td>19</td><td>∞</td><td>0</td><td>10</td></tr><tr><th>7</th><td>∞</td><td>∞</td><td>∞</td><td>8</td><td>∞</td><td>10</td><td>0</td></tr></table> Cạnh (2,4) có trọng số 18 nên: a[2][4] = a[4][2] = 18 Giữa đỉnh 5 và 7 không có cạnh nào nên: a[5][7] = a[7][5] = inf	a	1	2	3	4	5	6	7	1	0	3	∞	∞	∞	10	∞	2	3	0	6	18	∞	∞	∞	3	∞	6	0	15	12	∞	∞	4	∞	18	15	0	2	19	8	5	∞	∞	12	2	0	∞	∞	6	10	∞	∞	19	∞	0	10	7	∞	∞	∞	8	∞	10	0
a	1	2	3	4	5	6	7																																																										
1	0	3	∞	∞	∞	10	∞																																																										
2	3	0	6	18	∞	∞	∞																																																										
3	∞	6	0	15	12	∞	∞																																																										
4	∞	18	15	0	2	19	8																																																										
5	∞	∞	12	2	0	∞	∞																																																										
6	10	∞	∞	19	∞	0	10																																																										
7	∞	∞	∞	8	∞	10	0																																																										
	<table><tr><th>a</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th><th>7</th></tr><tr><th>1</th><td>0</td><td>4</td><td>∞</td><td>∞</td><td>∞</td><td>10</td><td>∞</td></tr><tr><th>2</th><td>3</td><td>0</td><td>6</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td></tr><tr><th>3</th><td>∞</td><td>∞</td><td>0</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td></tr><tr><th>4</th><td>∞</td><td>18</td><td>15</td><td>0</td><td>∞</td><td>∞</td><td>∞</td></tr><tr><th>5</th><td>∞</td><td>∞</td><td>12</td><td>2</td><td>0</td><td>∞</td><td>∞</td></tr><tr><th>6</th><td>∞</td><td>∞</td><td>∞</td><td>19</td><td>∞</td><td>0</td><td>∞</td></tr><tr><th>7</th><td>∞</td><td>∞</td><td>∞</td><td>8</td><td>∞</td><td>10</td><td>0</td></tr></table> Cung 1→2 có trọng số 4: a[1][2] = 4 Cung 2→1 có trọng số 3: a[2][1] = 3 Cung 4→6 không tồn tại: a[4][6] = inf	a	1	2	3	4	5	6	7	1	0	4	∞	∞	∞	10	∞	2	3	0	6	∞	∞	∞	∞	3	∞	∞	0	∞	∞	∞	∞	4	∞	18	15	0	∞	∞	∞	5	∞	∞	12	2	0	∞	∞	6	∞	∞	∞	19	∞	0	∞	7	∞	∞	∞	8	∞	10	0
a	1	2	3	4	5	6	7																																																										
1	0	4	∞	∞	∞	10	∞																																																										
2	3	0	6	∞	∞	∞	∞																																																										
3	∞	∞	0	∞	∞	∞	∞																																																										
4	∞	18	15	0	∞	∞	∞																																																										
5	∞	∞	12	2	0	∞	∞																																																										
6	∞	∞	∞	19	∞	0	∞																																																										
7	∞	∞	∞	8	∞	10	0																																																										

3.2.1.2. Danh sách kề

Để tối ưu hóa không gian bộ nhớ và tăng tốc độ thực thi cho thuật toán Dijkstra, đồ án triển khai cấu trúc dữ liệu danh sách kề tự định nghĩa. Khác với

ma trận kề tiêu tốn $O(V^2)$ không gian, danh sách kề chỉ tập trung lưu trữ các liên kết thực tế giữa các đỉnh, giúp tối thiểu hoá các bước duyệt đồ thị.

Trong chương trình, cấu trúc danh sách kề (`struct adjlist`) của một đỉnh được định nghĩa bao gồm các thành phần:

- Mảng một chiều `e[N]` lưu trữ thông tin của các cạnh kề là cấu trúc `cost` cho biết cặp dữ liệu: đỉnh kề (`v`) và trọng số cạnh kề (`w`).
- Biến số nguyên `size` giúp quản lý kích thước danh sách.
- Phương thức khởi tạo danh sách kề là hàm khởi tạo mặc định được định nghĩa sẵn bên trong cấu trúc dữ liệu. Phương thức này tự động kích hoạt ngay khi khai báo một đối tượng thuộc kiểu `adjlist`, thiết lập giá trị kích thước ban đầu của danh sách kề về bằng 0.

```
struct cost{
    int v;    // Chỉ số đỉnh
    float w;  // Khoảng cách
};

// Cấu trúc danh sách kề
struct adjlist{
    cost e[N]; // Danh sách các cạnh kề
    int size;  // Kích thước danh sách

    adjlist(){
        size = 0;
    }
};
```

Hình 5: Cấu trúc dữ liệu danh sách kề

Để quản lý danh sách kề cho mọi đỉnh, ta khai báo mảng một chiều các danh sách kề `adjlist adj[N]`. Với `adj[i]` là danh sách kề của đỉnh `i`.

Để vận hành cấu trúc này, hệ thống triển khai các hàm hỗ trợ sau:

STT	Hàm	Nguyên mẫu hàm	Thuật toán
1	Thêm một phần tử vào danh sách	<code>void push(adjlist &a, cost x)</code>	Gán phần tử cuối <code>e[a.size] = x</code> và tăng <code>a.size++</code>
2	Xoá hàng loạt danh sách kề	<code>void free_adjlist(adjlist a[])</code>	for <code>i = 1 → n</code> : gán kích thước <code>a[i].size = 0</code>

3.2.2. Cấu trúc dữ liệu hàng đợi ưu tiên - Heap

Để tối ưu hóa quá trình tính toán của thuật toán Dijkstra, cần phải triển khai cấu trúc dữ liệu hàng đợi ưu tiên dựa trên mô hình Min-Heap (cây nhị phân hướng gốc nhỏ nhất). Trong bộ nhớ máy tính, Min-Heap được tổ chức chặt chẽ theo các quy ước sau:

- Phần tử nằm ở gốc cây ($i = 1$) luôn luôn lưu trữ đỉnh có nhãn khoảng cách tạm thời nhỏ nhất ($d[u]$ bé nhất) trong số các đỉnh chưa được cố định.
- Với mọi nút trên cây, giá trị khoảng cách của nút cha luôn nhỏ hơn hoặc bằng giá trị khoảng cách của các nút con.

Trong thực tế lập trình C++, cấu trúc dữ liệu Min-Heap (`struct heap`) của chương trình được tổ chức thông qua các thành phần sau:

- Con trỏ `que` thuộc kiểu dữ liệu `cost` (được định nghĩa ở mục 3.2.1.2), con trỏ này sẽ được cấp phát bộ nhớ, trở thành cấu trúc mảng một chiều. Trong đó, `que[1]` đóng vai trò là gốc của Min-Heap, đồng thời là giá trị đứng đầu hàng đợi ưu tiên, là phần tử có giá trị khoảng cách (w) bé nhất.
- Số nguyên `size` đóng vai trò quản lý kích thước hàng đợi.
- Trong `struct heap` tích hợp sẵn phương thức khởi tạo (`heap`) cấp phát bộ nhớ đệm ban đầu cho hàng đợi, và phương thức hủy (`~heap`) đảm nhận nhiệm vụ giải phóng toàn bộ vùng nhớ động ngay khi dùng xong.

```
struct heap{
    cost *que; // Mảng lưu hàng đợi ưu tiên
    int size; // Kích thước hiện tại của mảng
    // Khởi tạo hàng đợi ngay khi khai báo
    heap(int maxSize){
        que = new cost[maxSize + 1];
        size = 0;
    }
    // Giải phóng bộ nhớ khi dùng xong heap
    ~heap(){
        delete[] que;
    }
};
```

Hình 6: Cấu trúc dữ liệu hàng đợi ưu tiên heap

Để vận hành, quản lý và thực hiện các thao tác cơ bản trên Heap, nhóm em đã xây dựng một hệ thống các hàm chức năng độc lập. Các hàm này chịu trách nhiệm từ việc khởi tạo, cấp phát bộ nhớ cho đến việc duy trì cấu trúc tối ưu của Heap trong suốt quá trình giải thuật Dijkstra:

STT	Hàm	Nguyên mẫu hàm	Thuật toán
1	Hàm kiểm tra hàng đợi rỗng	<code>bool isHeapEmpty(heap h);</code>	Kiểm tra xem kích thước của hàng đợi có bằng 0 hay không.
2	Hàm xử lý Heap	<code>void min_heap(heap &h, int i);</code>	Hàm này sắp xếp và duy trì trật tự: giá trị khoảng cách của nút cha luôn nhỏ hơn các nút con. Bằng thuật toán <code>min_heap</code> đi xuống với gốc là i: - Xét 3 nút: nút gốc i, nút con $i*2$, nút con $i*2+1$. - Nếu nút con m của i có giá trị nhỏ hơn nó, thì ta đổi vị trí nút con m với nút i. Sau đó thực hiện <code>min-heap</code> với gốc là nút m (với điều kiện m không phải nút lá).
3	Thêm phần tử vào hàng đợi ưu tiên	<code>void push(heap &h, cost x);</code>	Thêm phần tử vào hàng đợi ưu tiên. Thuật toán: - B1: Đẩy nút mới x vào cuối mảng $\rightarrow x$ nằm ở nút lá (nút i). - B2: Tìm nút cha của i: $i/2$. - B3: Nếu nút i có khoảng cách nhỏ hơn nút cha của nó thì đổi chỗ 2 nút này. - B4: $i = i/2$. Lặp lại bước 2 đến khi i là nút gốc.
4	Lấy phần tử cao nhất trong hàng đợi	<code>cost top(heap h);</code>	Trả về nút có giá trị khoảng cách nhỏ nhất trong hàng đợi là nút gốc (nút số 1).
5	Xoá phần tử cao nhất trong hàng đợi	<code>void pop(heap &h);</code>	- B1: Xoá phần tử cao nhất hàng đợi. - B2: Lấy phần tử cuối gán cho phần tử đầu hàng đợi. - B3: Giảm kích thước hàng đợi và <code>min-heap</code> cho nút gốc số 1.

3.2.3. Cấu trúc dữ liệu lưu trữ kết quả tối ưu và truy vết lộ trình

a) Mảng lưu trữ chi phí ngắn nhất

Cấu trúc dữ liệu này chịu trách nhiệm lưu trữ trọng số nhỏ nhất của các đường đi trong suốt tiến trình tối ưu hóa của thuật toán.

- Đối với thuật toán Dijkstra, ta sử dụng mảng một chiều toàn cục:

```
float DIJKSTRA[N];
```

Trong đó, `DIJKSTRA[i]` lưu trữ giá trị độ dài đường đi ngắn nhất hiện tại tính từ đỉnh nguồn `s` đến đỉnh `i`. Ban đầu, mọi phần tử của mảng được khởi tạo bằng vô cùng lớn, ngoại trừ `DIJKSTRA[s] = 0`.

- Đối với thuật toán Floyd-Warshall, ta sử dụng mảng hai chiều toàn cục:

```
float FLOYD[N][N];
```

Trong đó, `FLOYD[i][j]` lưu trữ khoảng cách ngắn nhất từ đỉnh `i` đến đỉnh `j`. Mảng này ban đầu được sao chép từ ma trận kề gốc của đồ thị.

b) Mảng lưu trữ trạng thái duyệt

Khai báo trong mã nguồn: `bool kt[N];`

Ý nghĩa: Mảng một chiều kiểu `bool` này chỉ áp dụng riêng cho thuật toán Dijkstra. Phần tử `kt[i]` nhận giá trị `true` nếu đỉnh `i` đã được thuật toán chọn ra từ Min-Heap và cố định nhãn tối ưu (tối ưu toàn cục); ngược lại sẽ nhận giá trị `false` nếu đỉnh đó vẫn đang trong trạng thái chưa được tối ưu. Việc duy trì mảng này giúp chương trình không lặp lại thao tác thư giãn cạnh trên các đỉnh đã tối ưu và tránh chu trình vô hạn.

c) Hệ thống dữ liệu truy vết lộ trình (Tracing)

Để khôi phục được chi tiết từng đỉnh trên đường đi, cơ chế truy vết được cài đặt dựa tùy theo từng thuật toán:

- Mảng truy vết Dijkstra là mảng một chiều toàn cục:

```
int DIJKSTRA_TRACE[N];
```

Giá trị `DIJKSTRA_TRACE[v] = u` có nghĩa là đỉnh `u` là đỉnh đứng ngay trước đỉnh `v` trên lộ trình tối ưu từ nguồn đến `v`. Khi cần xuất đường đi từ `s` đến `t`, chương trình sẽ chạy vòng lặp truy vết ngược từ `t` về `s` thông qua mảng `DIJKSTRA_TRACE`.

- Ma trận truy vết Floyd-Warshall là một mảng hai chiều toàn cục:

```
int FLOYD_TRACE[N][N];
```

Giá trị `FLOYD_TRACE[i][j]` lưu trữ đỉnh kế tiếp phải đi sau đỉnh *i* trên lộ trình tối ưu để đến được đích *j*.

3.2.4. Cấu trúc dữ liệu quản lý các đối tượng đồ họa

Bên cạnh việc lưu trữ đường đi ngắn nhất trên đồ thị bằng ma trận kề và mảng truy vết lộ trình, hệ thống phần mềm còn triển khai các mô hình cấu trúc dữ liệu đồ họa để phục vụ cho chức năng tương tác và hiển thị trên màn hình. Dưới đây là chi tiết các cấu trúc dữ liệu thành phần chịu trách nhiệm quản lý giao diện trực quan hóa của đồ án:

- Cấu trúc dữ liệu đồ họa dành cho đỉnh của đồ thị
 - Chỉ số đỉnh (id): là số hiệu của đỉnh tương ứng với dữ liệu đồ thị.
 - Vị trí đỉnh (position): được biểu diễn bằng cấu trúc tọa độ hai chiều `Vector2` (gồm hai thành phần số thực *x* và *y*) của thư viện `Raylib`, xác định vị trí tâm của nút trên màn hình giả lập.
 - Vận tốc đỉnh (veloc): là một vector hai chiều kiểu `Vector2`, đại diện cho vận tốc tức thời và hướng dịch chuyển của đỉnh. Thuộc tính này phục vụ cho việc tính toán các hiệu ứng vật lý động như lực đẩy giữa các nút, giúp đồ thị tự động giãn cách đều và đẹp mắt.
 - Cờ trạng thái kéo thả (isDragging): là biến logic xác định xem người dùng có đang đè giữ chuột trái và kéo nút di chuyển trên màn hình hay không, làm cơ sở để chương trình cập nhật lại tọa độ đỉnh theo thời gian thực.
 - Cờ trạng thái cố định (isFixed): là biến logic dùng để khóa đỉnh tại một vị trí cố định trên giao diện, ngăn không cho các thuật toán vật lý hay lực đẩy đồ thị làm xê dịch tọa độ mong muốn của người dùng.

```
struct Node{
    int id;           // Chỉ số đỉnh
    Vector2 position; // Vị trí đỉnh
    Vector2 veloc;    // Vận tốc đỉnh
    bool isDragging;  // Đỉnh có đang được kéo
    bool isFixed;     // Đỉnh có đang cố định
};
```

Hình 7: Cấu trúc dữ liệu đồ họa dành cho đỉnh

- Cấu trúc dữ liệu đồ hoạ dành cho cạnh của đồ thị
 - Đỉnh nguồn (src) và đỉnh đích (dest): là cặp chỉ số nguyên biểu thị hai đầu mút của cạnh. Với đồ thị có hướng, cặp đỉnh này xác định rõ hướng của cung: đi từ src đến dest.
 - Trọng số cạnh (weight): lưu trữ giá trị chi phí của tuyến đường kết nối dưới dạng số thực float. Giá trị này được sử dụng vào công việc vẽ đồ thị có trọng số của chương trình.
 - Cờ trạng thái lộ trình (isPath): là biến logic đóng vai trò trực quan hóa kết quả. Sau khi thuật toán tìm ra đường đi ngắn nhất, các cạnh nằm trên lộ trình tối ưu sẽ được bật cờ này lên giá trị true, kích hoạt đổi màu cho cạnh, giúp người dùng nhận diện tuyến đường tối ưu một cách trực quan.
 - Điểm giữa của cạnh (mid): được biểu diễn bằng cấu trúc tọa độ hai chiều Vector2, lưu trữ tọa độ điểm giữa của cạnh, được dùng để điều chỉnh cạnh thành đường cong Bezier trong trường hợp cạnh bị đè lên các đỉnh khác, giúp đồ thị được hiển thị rõ ràng hơn.

```
struct Edge{  
    int src;           // Đỉnh nguồn  
    int dest;          // Đỉnh đích  
    float weight;      // Trọng số cạnh  
    bool isPath;       // Cạnh có thuộc đường đi  
    Vector2 mid;        // Điểm giữa của cạnh  
};
```

Hình 8: Cấu trúc dữ liệu đồ hoạ dành cho cạnh

3.3. Thuật toán

Trên cơ sở cấu trúc dữ liệu đồ thị đã được thiết lập, nhóm em triển khai hai thuật toán Dijkstra và Floyd-Warshall để giải quyết bài toán tìm đường đi ngắn nhất.

3.3.1. Cài đặt thuật toán Dijkstra

Thuật toán Dijkstra tìm đường đi ngắn nhất từ một đỉnh nguồn đến các đỉnh khác, bao gồm hai giai đoạn chính: tìm và dán nhãn đỉnh có chi phí tạm thời nhỏ nhất, tối ưu hoá chi phí cho các đỉnh xung quanh.

Đầu vào cho thuật toán Dijkstra bao gồm: số đỉnh n , danh sách kề $a[N]$ của đồ thị, đỉnh bắt đầu s . Kết quả của thuật toán là mảng lưu khoảng cách $d[N]$ và mảng truy vết $g[N]$ được khai báo toàn cục.

Nhằm đảm bảo tính chính xác, chương trình sẽ tiến hành kiểm tra điều kiện hợp lệ của dữ liệu đầu vào, đảm bảo đồ thị không chứa cạnh có trọng số âm trước khi giải thuật Dijkstra.

Các bước triển khai thuật toán Dijkstra:

Cách 1 (không dùng hàng đợi ưu tiên):

- Bước 1: Với $i = 1 \dots n$, $d[i] = \inf(\infty)$, sau đó gán $d[s] = 0$
Khởi tạo mảng lưu trạng thái duyệt $kt[i] = 0$ (chưa duyệt)
- Bước 2: Duyệt $i = 1 \dots n$: gán $u = i$ nếu $kt[i] == 0$ và $d[i]$ nhỏ nhất
Nếu không tìm được u nào thỏa mãn thì dừng thuật toán
- Bước 3: Dán nhãn cho đỉnh u : $kt[u] = \text{true}$
- Bước 4: Duyệt qua tất cả các đỉnh v kề với u và chưa được dán nhãn:
Nếu $d[u] + a[u][v] < d[v]$:
 Cập nhật lại $d[v] = d[u] + a[u][v]$
 Gán đỉnh kề trước của v là u : $g[v] = u$
Quay lại bước 2

Cách 2 (có dùng hàng đợi ưu tiên):

- Bước 1: Với $i = 1 \dots n$, $d[i] = \inf(\infty)$, sau đó gán $d[s] = 0$
Khởi tạo mảng lưu trạng thái duyệt $kt[i] = 0$ (chưa duyệt)
Đẩy đỉnh bắt đầu s và giá trị $d[s]$ của nó vào hàng đợi ưu tiên q
- Bước 2: Nếu hàng đợi q chưa rỗng: gán $u = \text{top}(q).v$ và xóa u khỏi q
Else dừng thuật toán
- Bước 3: Nếu đã dán nhãn đỉnh u ($kt[u] == \text{true}$) thì quay lại bước 2
Else dán nhãn cho đỉnh u : $kt[u] = \text{true}$
- Bước 4: Duyệt qua tất cả các đỉnh v kề với u và chưa được dán nhãn:
Nếu $d[u] + a[u][v] < d[v]$:
 Cập nhật lại $d[v] = d[u] + a[u][v]$
 Gán đỉnh kề trước của v là u : $g[v] = u$
 Đẩy cặp giá trị $\{ v, d[v] \}$ vào hàng đợi q
Quay lại bước 2

Để so sánh hiệu năng hai cách trên, ta cần đánh giá dựa trên độ phức tạp của thuật toán trong mỗi cách:

- Đối với cách đầu tiên, thuật toán thực hiện vòng lặp duyệt qua n lượt tương ứng với n đỉnh cần dán nhãn cố định. Tại mỗi lượt lặp, chương trình phải quét tuần tự qua n phần tử để tìm ra đỉnh u chưa dán nhãn có giá trị $d[u]$

nhỏ nhất, tiêu tốn $O(n)$ thời gian. Song song với đó, quá trình giãn cạnh sẽ duyệt qua tổng cộng m cạnh của đồ thị trong suốt toàn bộ thuật toán với chi phí $O(m)$. Do đó, độ phức tạp thời gian tổng thể của Cách 1 là $O(n^2 + m)$. Vì $0 \leq m \leq n^2$, biểu thức Big-O trên được tối giản thành $O(n^2)$.

- Đối với cách thứ hai, thuật toán cũng vận hành qua các lượt duyệt để dán nhãn cho các nút. Tuy nhiên, tại mỗi lượt, thay vì duyệt tuần tự, hệ thống sử dụng cấu trúc dữ liệu Min-Heap để lập tức trích xuất và xóa đỉnh u ra khỏi hàng đợi, tiêu tốn thời gian $O(\log n)$. Đối với bước giãn cạnh, hệ thống cũng xét qua tổng cộng m cạnh trên danh sách kề; nhưng tại mỗi lần tối ưu thành công, chương trình sẽ gọi hàm push để cập nhật lại cấu trúc Heap, tiêu tốn chi phí là $O(\log n)$ cho mỗi thao tác. Do đó, tổng độ phức tạp của Cách 2 là $O((n + m) \log n)$.

Như vậy, cách thứ hai có sử dụng Min-Heap nên độ phức tạp thời gian nhỏ hơn, giúp tối ưu thời gian thực thi của thuật toán Dijkstra.

```
bool dijkstra(int s, adjlist a[], int n) {
    // Kiểm tra xem có cạnh âm hay không
    for (int i=1; i<=n; i++){
        for (int j=0; j<a[i].size; j++){
            if (a[i].e[j].w < 0.0f) return 0;
        }
    }
    heap q(numEdges);
    bool kt[N];
    // Khởi tạo giá trị ban đầu
    for (int i=1; i<=n; i++){ DIJKSTRA[i] = inf; kt[i] = 0; }
    DIJKSTRA[s] = 0; DIJKSTRA_TRACE[s] = s;
    push(q, (cost){s,0});
    int u, i;
    float w;

    while(!isHeapEmpty(q)){
        u = top(q).v; // Tìm đỉnh cần dán nhãn
        pop(q);
        if (kt[u]) continue;
        kt[u] = 1; // Dán nhãn
        // Giãn cạnh
        for(int j=0; j<a[u].size; j++) {
            i = a[u].e[j].v;
            w = a[u].e[j].w;
            if(!kt[i] && DIJKSTRA[i] > DIJKSTRA[u] + w){
                DIJKSTRA[i] = DIJKSTRA[u] + w;
                DIJKSTRA_TRACE[i] = u;
                push(q, (cost){i, DIJKSTRA[i]});
            }
        }
    }
    return 1;
}
```

Hình 9: Cài đặt hàm xử lý thuật toán Dijkstra

Sau khi giải thuật Dijkstra kết thúc, hệ thống dựa vào mảng lưu đỉnh kế trước $g[N]$ để chạy vòng lặp truy ngược và khôi phục lộ trình từ đích về nguồn. Thuật toán truy vết được triển khai cụ thể như sau:

- Bước 1: Khai báo stack d (thực hiện cơ chế FILO – First In Last Out)
Khởi tạo $u = t$ (đỉnh đích)
- Bước 2: Đẩy u vào stack d
Gán $u = g[t]$
- Bước 3: Nếu $u == s$ thì đẩy s vào d và trả về kết quả là stack d
Else quay lại bước 2

```
void get_path_dijkstra(int s, int t, int path[], int &n) {  
    n = 0;  
    int u = t;  
    stack<int> d;  
    d.push(t);  
    while(u != s){ u = DIJKSTRA_TRACE[u]; d.push(u); }  
    while(!d.empty()){  
        path[n++] = d.top();  
        d.pop();  
    }  
}
```

Hình 10: Cài đặt hàm truy vết đường đi ngắn nhất đối với thuật toán Dijkstra

3.3.2. Cài đặt thuật toán Floyd

Thuật toán Floyd thực hiện tính toán đường đi ngắn nhất giữa mọi cặp đỉnh với đầu vào là số đỉnh n và ma trận kề $a[N][N]$. Sau đó trả về kết quả trên ma trận kề $a[][]$ và ma trận truy vết $g[N][N]$.

Quy trình thực thi chi tiết của thuật toán Floyd:

- Bước 1: for ($i, j = 1 \rightarrow n$) khởi tạo ma trận truy vết $g[i][j] = j$
- Bước 2: for ($k = 1 \rightarrow n$): for ($i = 1 \rightarrow n$): for ($j = 1 \rightarrow n$):
 Nếu $a[i][k] + a[k][j] < a[i][j]$ thì:
 Gán $a[i][j] = a[i][k] + a[k][j]$
 Gán $g[i][j] = g[i][k]$
- Bước 3: Kiểm tra xem đồ thị có chu trình âm hay không:
 for ($i = 1 \rightarrow n$): Nếu $a[i][i]$ âm thì đồ thị có chu trình âm
 Nếu không tồn tại $a[i][i]$ âm thì kết quả $a[][]$ đúng

```

bool floyd(float a[][N], int n) {
    for (int i=1; i<=n; i++) for (int j=1; j<=n; j++) FLOYD[i][j] = a[i][j];
    for (int i=1; i<=n; i++) for (int j=1; j<=n; j++) FLOYD_TRACE[i][j] = j;

    for (int k=1; k<=n; k++)
        for (int i=1; i<=n; i++)
            for (int j=1; j<=n; j++)
                if (FLOYD[i][k] + FLOYD[k][j] < FLOYD[i][j]){
                    FLOYD[i][j] = FLOYD[i][k] + FLOYD[k][j];
                    FLOYD_TRACE[i][j] = FLOYD_TRACE[i][k];
                }

    // kiểm tra xem có chu trình âm hay không
    for (int i=1; i<=n; i++)
        if (FLOYD[i][i] < 0.0f) return 0;
    return 1;
}

```

Hình 11: Cài đặt hàm xử lý thuật toán Floyd

Dựa trên cấu trúc ba vòng lặp for lồng nhau và hai ma trận chiều lưu trữ chi phí, thuật toán Floyd sở hữu độ phức tạp thời gian là $O(n^3)$ và độ phức tạp không gian là $O(n^2)$. Dù chi phí tính toán bậc ba tương đối lớn, thuật toán này lại sở hữu ưu điểm xây dựng ma trận đường đi toàn cục; sau khi hoàn tất giai đoạn tiền xử lý, hệ thống có thể lập tức phản hồi và truy vết lộ trình giữa hai cặp đỉnh bất kỳ.

Sau khi thuật toán hoàn thành, nếu kết quả hợp lệ (đồ thị không có chu trình âm) thì ta có thể truy vết đường đi ngắn nhất bằng mảng hai chiều $g[][]$. Thuật toán truy vết được thực hiện như sau:

- Bước 1: Gán $u = s$ (đỉnh nguồn)
- Bước 2: Thêm đỉnh u vào cuối lộ trình hiện tại
- Bước 3: Nếu $u \neq t$ (đỉnh đích) thì gán $u = g[u][t]$, quay lại bước 2
Else dừng và trả về kết quả

```

void get_path_floyd(int s, int t, int path[], int &n) {
    n = 0;
    int u = s;
    path[n++] = s;
    while (u != t) {
        u = FLOYD_TRACE[u][t];
        path[n++] = u;
    }
}

```

Hình 12: Cài đặt hàm truy vết đường đi ngắn nhất đối với thuật toán Floyd

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức chương trình

4.1.1. Tổ chức tệp tin

Chương trình được thiết kế theo kiến trúc phân lớp mô-đun hóa rõ ràng, phân tách rõ ràng giữa tầng quản lý dữ liệu hệ thống, tầng xử lý thuật toán và tầng hiển thị giao diện đồ họa. Mã nguồn dự án được tổ chức thành các file thành phần như sau:

- File `main.cpp`: Thực hiện khởi tạo cửa sổ ứng dụng thông qua hàm `InitWindow()`. File này còn chứa vòng lặp chính của chương trình, liên tục cập nhật màn hình đồ họa trong thời gian thực.
- File `header.h`: Nơi khai báo và lưu trữ tập trung tất cả các thư viện chuẩn và định nghĩa các hằng số cấu hình toàn cục, các cấu trúc dữ liệu cốt lõi.
- File `algo.h`: Tập mã nguồn lưu trữ chi tiết cài đặt của thuật toán Dijkstra và Floyd, đồng thời định nghĩa các cấu trúc dữ liệu phục vụ cho thuật toán chính: `cost`, `adjlist` và `heap`.
- File `physics.h`: Cài đặt các hàm toán học vi phân để tính toán vector vận tốc và vị trí thời gian thực cho các đỉnh, chịu trách nhiệm thực thi giả lập động lực học cho đồ thị, được cấu trúc gồm các hàm chính:

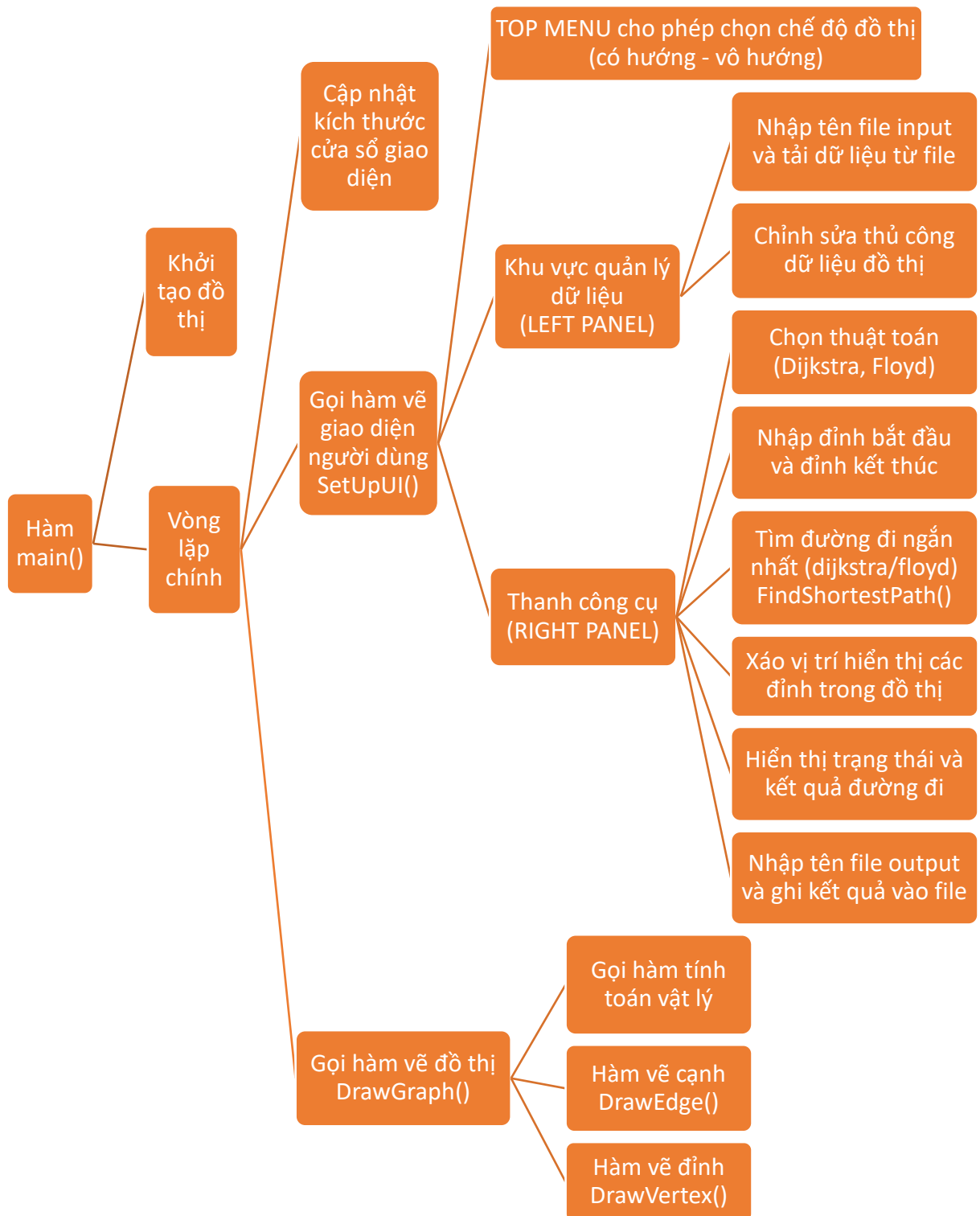
STT	Hàm	Chức năng
1	<code>GetDist()</code>	Tính toán khoảng cách giữa 2 điểm trong mặt phẳng toạ độ
2	<code>GetDotProduct()</code>	Tính toán tích vô hướng của hai vector
3	<code>GetMidPoint()</code>	Tính toán vị trí trung điểm của cạnh
4	<code>Vertex_VerxCompulsion()</code>	Tính toán lực đẩy giữa hai đỉnh
5	<code>Vertex_EdgeCompulsion()</code>	Tính toán lực đẩy giữa đỉnh và cạnh (giữa đỉnh và điểm giữa của cạnh)
6	<code>Edge_EdgeCompulsion()</code>	Tính toán lực đẩy giữa hai cạnh có cùng hai đầu mút
7	<code>AttractiveForce()</code>	Tính toán lực kéo giữa hai đỉnh kề
8	<code>PhysicsCalc()</code>	Hàm tính toán tất cả thuộc tính vật lí

- File `graphics.h`: Xử lý giao diện đồ họa tương tác, bao gồm toàn bộ bộ cục giao diện bằng điều khiển UI thông qua thư viện Raygui. Tập tin được tổ chức thành các hàm chính:

STT	Hàm	Chức năng
1	<code>DrawVertex()</code>	Vẽ đỉnh đồ thị (là các hình tròn), tô màu cho các đỉnh được chọn làm đỉnh nguồn, đỉnh đích, đỉnh cố định
2	<code>DrawEdge()</code>	Vẽ cạnh của đồ thị (là các đường thẳng hoặc đường cong Bézier), tô màu cho các cạnh nằm trên đường đi
3	<code>EditGraphData()</code>	Hàm xử lý thao tác chỉnh sửa dữ liệu đồ thị bằng cách thủ công
4	<code>LoadStringFromFile()</code> <code>LoadGraphFromString()</code> <code>SaveFileFromString()</code>	Đọc dữ liệu đồ thị từ file thông qua chuỗi trung gian
5	<code>FindShortestPath()</code>	Hàm xử lý thuật toán chính, gọi hàm thực thi Dijkstra, Floyd và hàm truy vết đường đi ngắn nhất
6	<code>SetUpUI()</code>	Vẽ đồ họa giao diện có tương tác, bao gồm thanh quản lý dữ liệu và thanh công cụ
7	<code>DrawGraph()</code>	Vẽ đồ thị, gọi các hàm vẽ đỉnh, vẽ cạnh và hàm tính toán vật lý

4.1.2. Tổ chức luồng thực thi chương trình

Tiến trình vận hành của ứng dụng được điều phối đồng bộ thông qua các hàm chức năng được thiết lập tại hàm `main()` trong tệp mã nguồn `main.cpp`. Luồng thực thi được minh họa cụ thể trong sơ đồ dưới đây:



Hình 13: Sơ đồ tổ chức luồng thực thi tổng thể chương trình

4.2. Ngôn ngữ cài đặt

Ứng dụng được thiết kế và triển khai dựa trên bộ công cụ kỹ thuật cùng môi trường lập trình cụ thể bao gồm:

- Ngôn ngữ lập trình chính: Ứng dụng được triển khai hoàn toàn trên ngôn ngữ lập trình C++ nhờ vào khả năng tối ưu hóa hiệu năng và tốc độ xử lý vượt trội. Ngôn ngữ này cho phép chương trình can thiệp sâu vào việc quản lý bộ nhớ động, giúp tối ưu hóa các cấu trúc dữ liệu một cách chặt chẽ.
- Thư viện được sử dụng: Chương trình tích hợp bộ thư viện chuẩn của C/C++. Cụ thể, nhóm thư viện hệ thống: `<stdio.h>`, `<math.h>`, `<string.h>`, `<stack>` chịu trách nhiệm xử lý luồng nhập xuất thông tin, cấu trúc dữ liệu và các phép toán hình học. Bên cạnh đó, toàn bộ phần tương tác trực quan và menu đa năng được vận hành nhờ hai thư viện `<raylib.h>` và `<raygui.h>`.
- Môi trường phát triển (IDE): Ứng dụng được xây dựng trên môi trường Visual Studio Code, một trình soạn thảo mã nguồn hiện đại giúp quản lý các tệp cấu trúc một cách hệ thống.
- Trình biên dịch (Compiler): Mã nguồn được biên dịch trực tiếp bằng trình biên dịch MinGW nhằm đảm bảo ứng dụng sau khi đóng gói có thể khởi chạy mượt mà và ổn định.

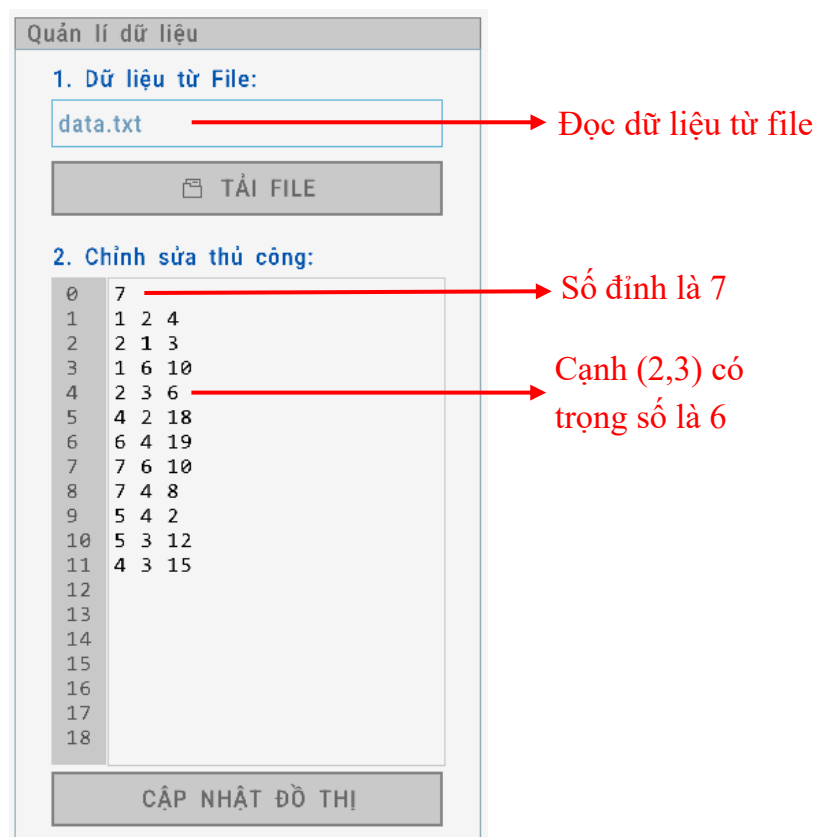
4.3. Kết quả

4.3.1. Giao diện chính của chương trình

Giao diện ứng dụng được xây dựng trên nền tảng thư viện đồ họa Raylib kết hợp với thư viện Raygui. Bố cục tổng thể được thiết kế theo cấu trúc phân khối chức năng, chia màn hình làm 3 khu vực chính giúp người dùng dễ dàng thao tác và quan sát trực quan:

- Khu vực đồ họa trên cùng:
 - Hiển thị đầy đủ thông tin định danh của đồ án bao gồm: Tên học phần (PBL1), tên đề tài, giảng viên hướng dẫn và sinh viên thực hiện.
 - Bên phải tích hợp nút chuyển đổi nhanh chế độ đồ thị: Vô hướng hoặc Có hướng, giúp hệ thống tự động thay đổi thuật toán vẽ và tính toán tương ứng.

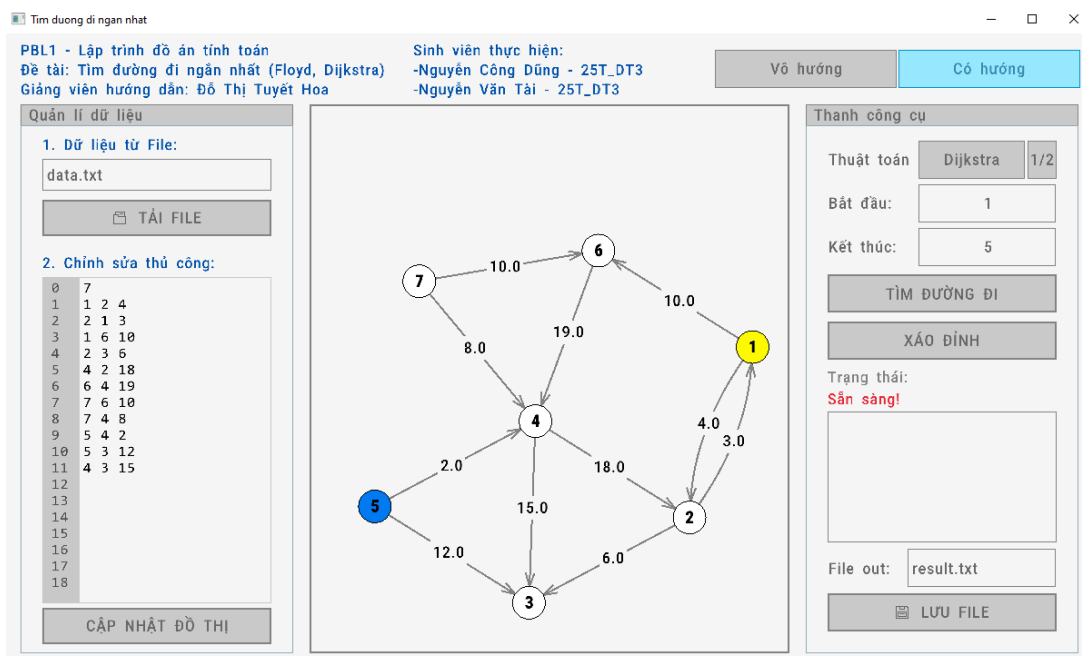
- Khu vực đồ họa hiển thị bên trái:
 - Chức năng đọc dữ liệu: Tích hợp ô nhập tên tệp dữ liệu và nút lệnh “TẢI FILE”, cho phép chương trình tự động đọc dữ liệu từ tệp tin bên ngoài.
 - Chức năng chỉnh sửa thủ công: Cung cấp một khung văn bản tương tác hiển thị dữ liệu đầu vào của đồ thị (hàng 0 là số đỉnh, các hàng tiếp theo là bộ ba số: <đỉnh nguồn> <đỉnh đích> <trọng số cạnh>). Người dùng có thể chỉnh sửa thủ công các thông số. Sau đó nhấn nút “CẬP NHẬT ĐỒ THỊ” để đồng bộ lại dữ liệu đồ thị.



Hình 14: Giao diện khu vực quản lý dữ liệu đầu vào của đồ thị

- Khu vực biểu diễn đồ thị ở giữa: Khu vực này cho phép người dùng tương tác trực tiếp bằng cách click giữ chuột trái để kéo thả di chuyển vị trí các đỉnh, click chuột trái hai lần để cố định đỉnh, hoặc cuộn con lăn chuột để zoom xa gần đồ thị.

- Khu vực đồ hoạ hiển thị bên phải:
 - Chọn cấu hình giải thuật: Cho phép người dùng chuyển đổi linh hoạt giữa hai thuật toán Dijkstra và Floyd thông qua hộp lựa chọn.
 - Chọn nhanh chỉ số đỉnh bắt đầu và kết thúc.
 - Nút “TÌM ĐƯỜNG ĐI” kích hoạt thuật toán để tìm và vẽ đường đi ngắn nhất.
 - Nút “XÁO ĐỈNH” hỗ trợ tái định vị ngẫu nhiên tọa độ các nút, tăng tính sinh động cho việc quan sát đồ thị.
 - Thông báo trạng thái: hiển thị các thông báo của hệ thống như "Sẵn sàng!" hoặc "LỖI: Không tìm thấy file!".
 - Xuất dữ liệu: nhập tên file và nút “LƯU FILE” để lưu lại kết quả lộ trình và chi phí đường đi sau khi tính toán.

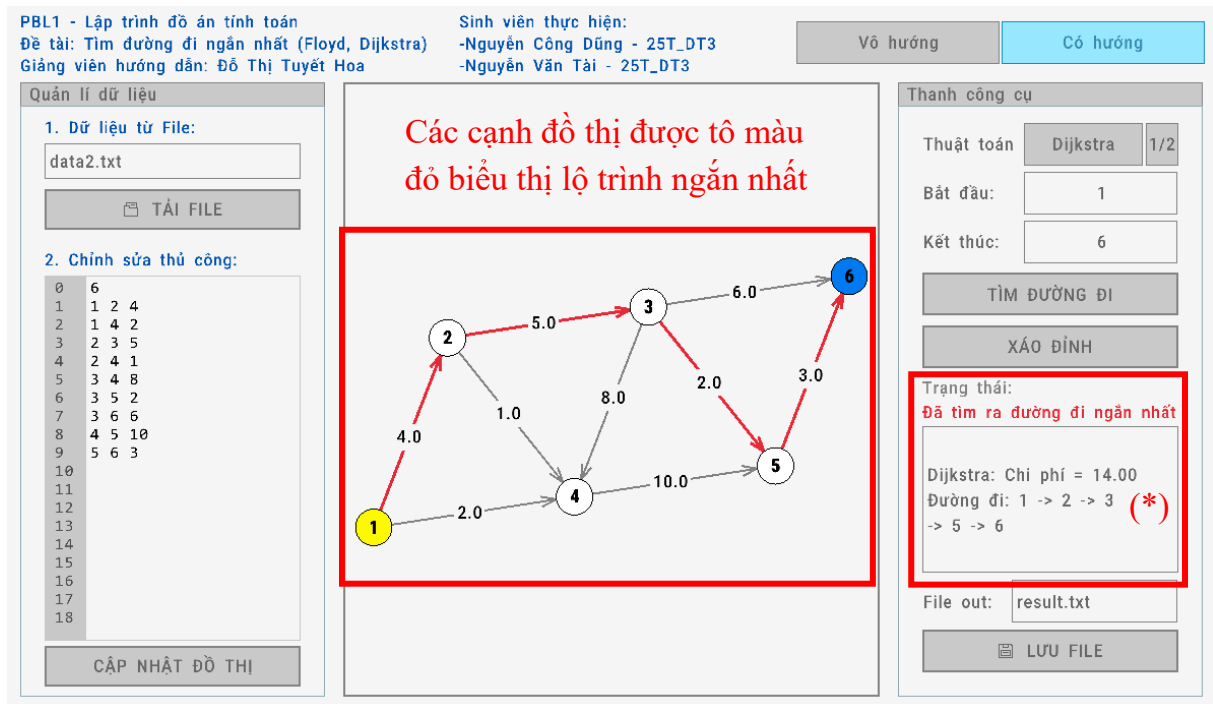


Hình 15: Giao diện tổng quan của chương trình khi vừa khởi chạy

4.3.2. Kết quả thực thi của chương trình

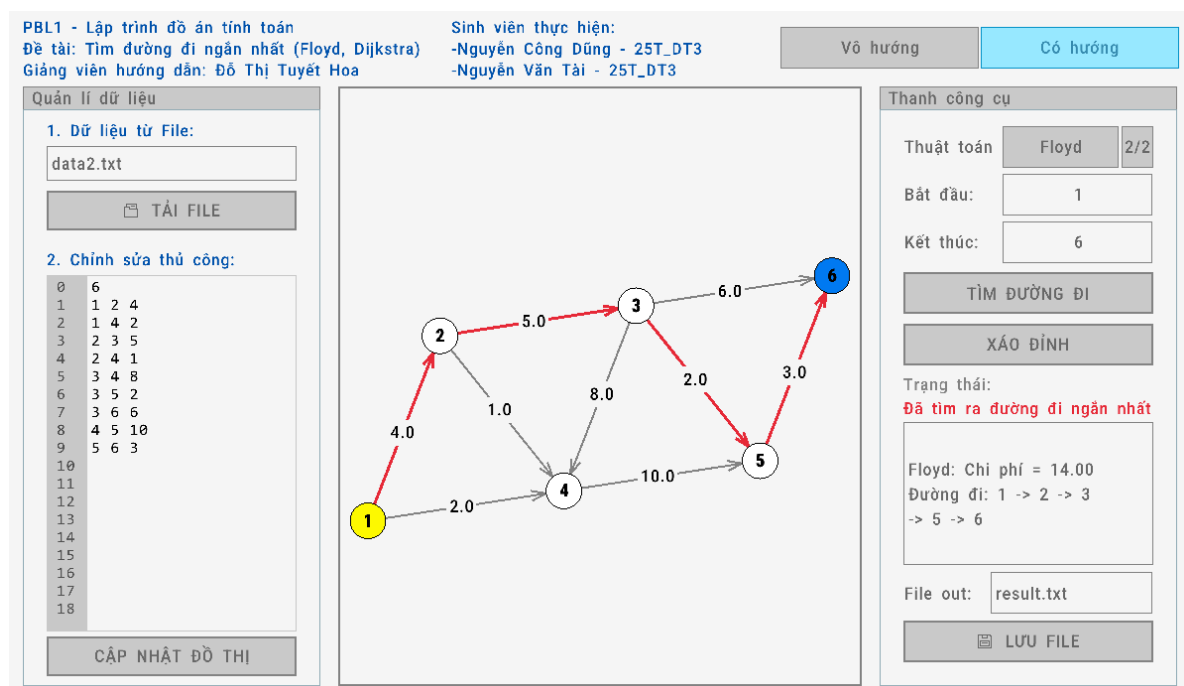
Tại giao diện của ứng dụng, sau khi người dùng nhấn kích hoạt nút “TÌM ĐƯỜNG ĐI”, hệ thống sẽ lập tức tiếp nhận tham số đầu vào là chỉ số đỉnh bắt đầu kết thúc tại thanh công cụ bên phải và thực thi thuật toán. Kết quả đường đi được hiển thị trực quan trên đồ thị (các cạnh trên đường đi được đổi màu), đồng thời chi phí và thứ tự các đỉnh trên đường đi (*) cũng được hiển thị trên thanh công cụ.

- Đối với thuật toán Dijkstra:



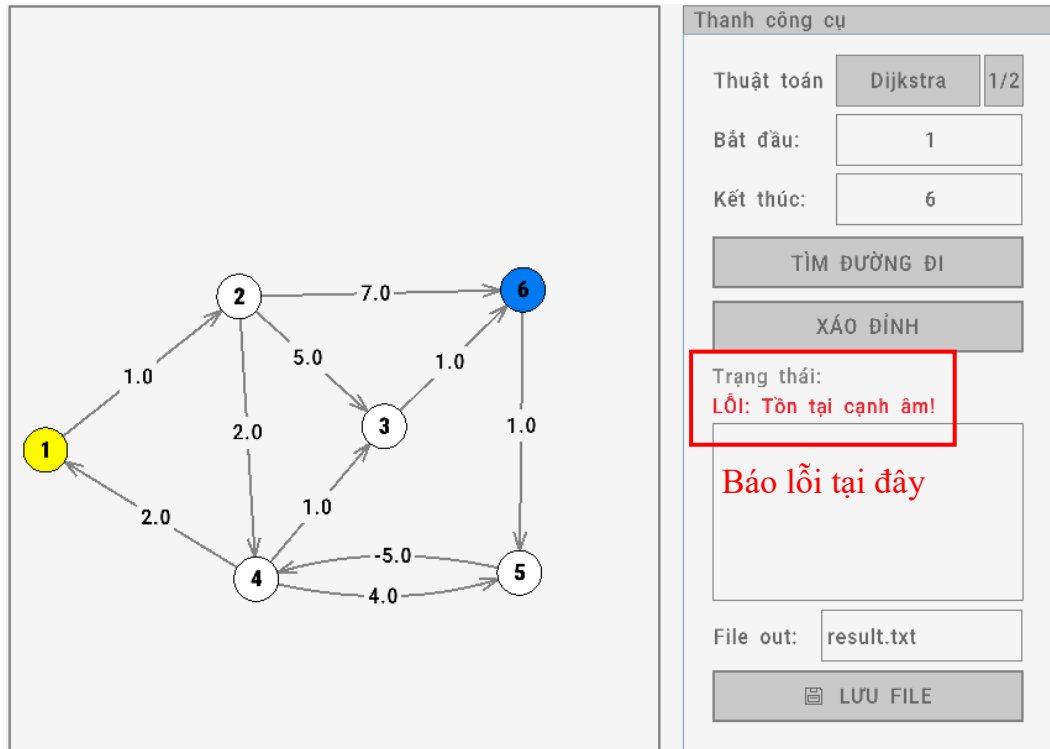
Hình 16: Kết quả đường đi ngắn nhất theo thuật toán Dijkstra

- Đối với thuật toán Floyd có cùng dữ liệu đầu vào:

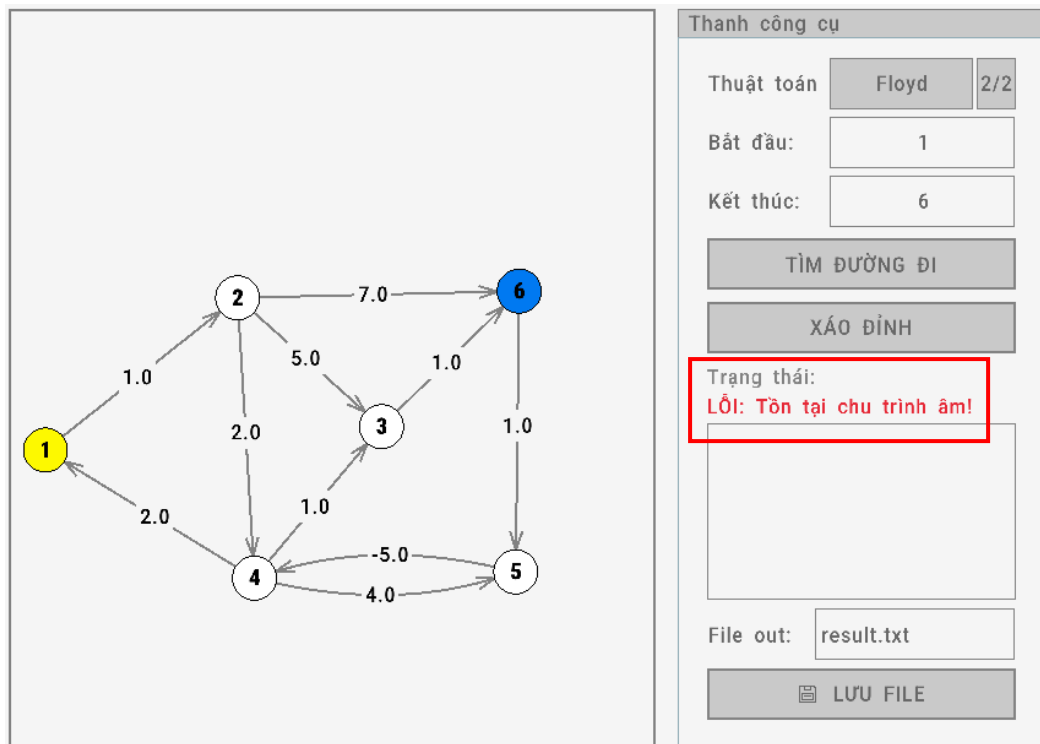


Hình 17: Kết quả đường đi ngắn nhất theo thuật toán Floyd

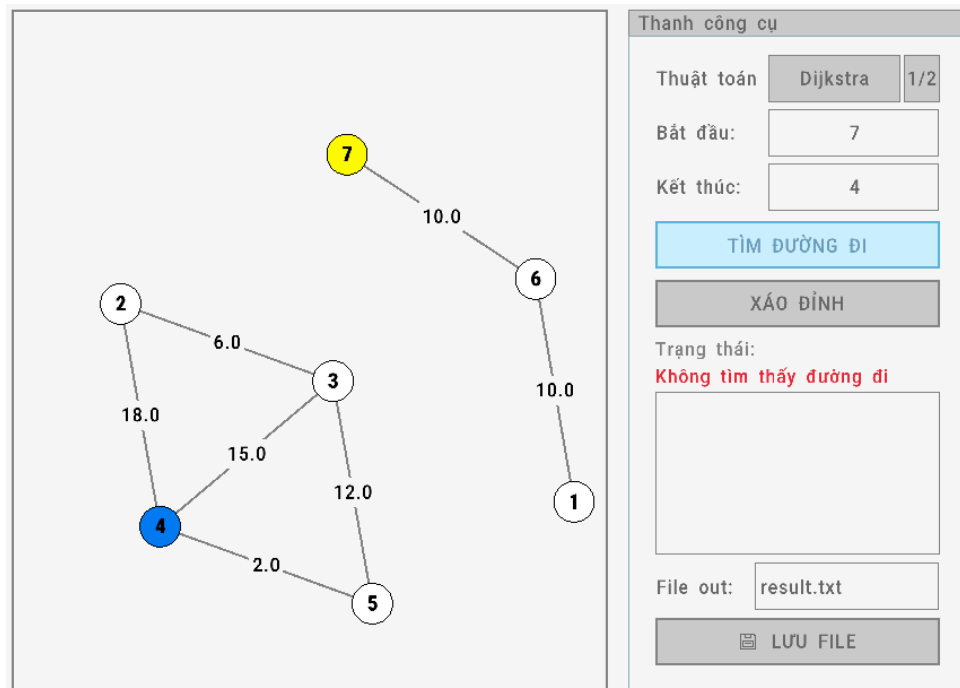
- Đối với các trường hợp không tìm thấy đường đi:



Hình 18: Báo lỗi thuật toán Dijkstra khi đồ thị tồn tại cạnh âm



Hình 19: Báo lỗi thuật toán Floyd khi đồ thị tồn tại chu trình âm



Hình 20: Chương trình không tìm thấy đường đi giữa hai đỉnh trên hai thành phần liên thông khác nhau của đồ thị

4.3.3. Nhận xét

Thông qua các kết quả thực nghiệm trên, hệ thống thể hiện sự ổn định và độ tin cậy cao trong nhiều trường hợp dữ liệu khác nhau.

- Về tính chính xác và hiệu năng của thuật toán:
 - Cả hai thuật toán Dijkstra và Floyd-Warshall đều được cài đặt chính xác, cho ra kết quả khoảng cách và chi tiết lộ trình ngắn nhất đồng nhất, tối ưu trên mọi bộ test chuẩn.
 - Việc tối ưu hóa cấu trúc dữ liệu danh sách kề và cấu trúc Min-Heap giúp chương trình vận hành an toàn với quy mô phần tử lớn.
 - Các bước tiền xử lý giúp kiểm soát chặt chẽ lỗi trường hợp biên như đồ thị không liên thông, chứa cạnh âm hay chu trình âm.
- Về tính năng và trải nghiệm người dùng:
 - Chức năng trực quan hóa đồ thị bằng thư viện raylib hoạt động hiệu quả: tọa độ các nút giao được phân bổ tự động bằng thuật toán lực co giãn, giúp các đỉnh và mũi tên hướng cạnh không bị đè lên nhau.

- Tính năng trình bày đồ thị có màu kết hợp làm nổi bật tuyến đường tối ưu giúp người dùng lập tức nhận diện kết quả một cách trực quan.
- Giao diện góp phần tối ưu hóa trải nghiệm người dùng nhờ việc tích hợp trực quan các ô nhập liệu và nút bấm chuyển đổi chức năng, giúp đơn giản hóa quy trình tương tác.

5. KẾT LUẬN

5.1. Kết quả đạt được

Sau một thời gian tìm hiểu và thực hiện đồ án PBL1, nhóm đã hoàn thành được các yêu cầu cơ bản của đề tài:

- Về kiến thức lập trình: Nhóm đã vận dụng được các kiến thức C++ cơ bản đã học vào một bài toán thực tế. Đồng thời, nhóm biết cách tổ chức chương trình thành các file .h và .cpp để dễ dàng quản lý đồ án.
- Về thuật toán: Cài đặt thành công và hiệu được luồng chạy của hai thuật toán Dijkstra và Floyd-Warshall. Đặc biệt với Dijkstra, nhóm đã tìm hiểu và cài đặt cấu trúc hàng đợi ưu tiên (Min-Heap), giúp tối ưu hiệu năng của thuật toán.
- Về giao diện: Nhóm tự nghiên cứu và sử dụng được thư viện Raylib để vẽ đồ thị ra màn hình. Kết quả in ra được trực quan hoá, giúp người dùng dễ dàng theo dõi.
- Về phần nâng cao: Nhóm cài đặt thêm một vài thuật toán giúp đồ thị tự động dẫn ra, tránh trường hợp các đối tượng bị đè lên nhau. Người dùng cũng có thể dùng chuột để điều chỉnh đồ thị.

5.2. Hạn chế

Dù chương trình đã cho ra kết quả đúng, nhưng do kiến thức còn hạn chế nên đồ án của nhóm vẫn còn một số khuyết điểm:

- Số đỉnh tối đa được khai báo sẵn $N = 100$, nếu người dùng nhập đồ thị có hơn 100 đỉnh thì chương trình không xử lý được.

- Thuật toán Floyd-Warshall sở hữu độ phức tạp thời gian lớn $O(n^3)$, dẫn đến việc tính toán dễ gây khựng nhịp trong vòng lặp đồ họa thời gian thực khi quy mô số đỉnh tăng lên quá lớn.
- Thao tác người dùng chưa tiện lợi vì hiện tại muốn có đồ thị thì người dùng phải nhập tay danh sách cạnh vào ô input. Chương trình chưa hỗ trợ tính năng click chuột ra màn hình để tự tạo đỉnh hay kéo chuột để nối cạnh trực tiếp.
- Hệ thống mới dừng lại ở việc hiển thị đồ thị tĩnh và đưa ra kết quả cuối cùng; chương trình chưa có chế độ chạy từng bước mô phỏng tiến trình duyệt nhân của Dijkstra để tăng tính giáo khoa cho ứng dụng.

5.3. Hướng phát triển

Nếu có thêm thời gian và kiến thức trong các học kỳ tới, nhóm dự định sẽ cải thiện chương trình theo các hướng sau:

- Học cách sử dụng cấp phát động dùng con trỏ hoặc dùng thư viện `std::vector` của C++ để lưu trữ đồ thị thay cho mảng tĩnh, giúp chương trình linh hoạt hơn và không bị giới hạn 100 đỉnh.
- Nghiên cứu và tích hợp các thuật toán tìm đường có khả năng xử lý cạnh trọng số âm (như thuật toán Bellman-Ford), đồng thời tối ưu hóa luồng tính toán của Floyd-Warshall bằng kỹ thuật đa luồng.
- Phát triển và hoàn thiện các tính năng tương tác chuột trực tiếp trên khung hiển thị, cho phép người dùng click để khởi tạo đỉnh mới, kéo thả chuột để thiết lập các cạnh nối và tùy chỉnh nhanh trọng số nhằm nâng cao trải nghiệm người dùng.
- Triển khai chế độ mô phỏng từng bước giúp trực quan hóa sinh động tiến trình duyệt và cập nhật hàng đợi ưu tiên của thuật toán Dijkstra.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Văn Hiệu, *Bài giảng Toán rời rạc: Bài toán đường đi*, Trường Đại học Bách khoa - Đại học Đà Nẵng, 2022.
- [2] Phan Thanh Tao, *Bài giảng Toán rời rạc: Chương 7. Các bài toán tối ưu trên đồ thị*, Trường Đại học Bách khoa - Đại học Đà Nẵng, 2012.
- [3] Đặng Thiên Bình, *Bài giảng Cấu trúc dữ liệu: Binary Search Trees*, Trường Đại học Bách khoa - Đại học Đà Nẵng, 2022.
- [4] Đặng Thiên Bình, *Bài giảng Cấu trúc dữ liệu: Heap Sort*, Trường Đại học Bách khoa - Đại học Đà Nẵng.

PHỤ LỤC

header.h

```
#ifndef HEADER_H
#define HEADER_H

#include <stdio.h>
#include <math.h>
#include <stack>
#include <raylib.h>
#include <string.h>
using namespace std;
#define RAYGUI_IMPLEMENTATION
#include "raygui.h"

float ratio = 1.0f;
float stdratio = 1.0f;
#define NODE_RADIUS 18.0f * stdratio
#define LINE_WEIGHT 2.0f
#define GRAPH_PANEL_MARGIN 30.0f
#define MAX_ROW 8192
#define MAX_COL 128
#define N 105
#define M 10005
#define inf 1e30

struct Node{
    int id;          // Chỉ số đỉnh
    Vector2 position; // Vị trí đỉnh
    Vector2 veloc;   // Vận tốc đỉnh
    bool isDragging; // Đỉnh có đang được kéo
    bool isFixed;    // Đỉnh có đang cố định
};

struct Edge{
    int src;        // Đỉnh nguồn
    int dest;       // Đỉnh đích
    float weight;   // Trọng số cạnh
    bool isPath;    // Cạnh có thuộc đường đi
    Vector2 mid;    // Điểm giữa của cạnh
};
```

```
int textScrollY = 0;
int textScrollX = 0;
int ptrX = 0;
int ptrY = 0;

int clickCount = 0;
float doubleClickTimer = 0;
int selectedIndex = -1;

#endif
```

algo.h

```
#include "header.h"

struct cost{
    int v;    // Chỉ số đỉnh
    float w;  // Khoảng cách
};

// Cấu trúc danh sách kề
struct adjlist{
    cost e[N]; // Danh sách các cạnh kề
    int size;  // Kích thước danh sách

    adjlist(){
        size = 0;
    }
};

void push(adjlist &a, cost x)
{
    a.e[a.size] = x;
    a.size++;
}

void free_adjlist(adjlist a[])
{
    for (int i=1; i<=numNodes; i++) a[i].size = 0;
}

// Định nghĩa heap - priority_queue
```

```
struct heap{
    cost *que; // Mảng lưu hàng đợi ưu tiên
    int size; // Kích thước hiện tại của mảng
    // Khởi tạo hàng đợi ngay khi khai báo
    heap(int maxSize){
        que = new cost[maxSize + 1];
        size = 0;
    }
    // Giải phóng bộ nhớ khi dùng xong heap
    ~heap(){
        delete[] que;
    }
};

bool isHeapEmpty(heap &h)
{
    return !h.size;
}

void swap(cost &a, cost &b)
{
    cost c = a;
    a = b; b = c;
}

void min_heap(heap &h, int i)
{
    int m = i;
    int l = i*2;
    int r = i*2 + 1;
    if (l <= h.size && h.que[l].w < h.que[m].w) m = l;
    if (r <= h.size && h.que[r].w < h.que[m].w) m = r;
    if (m != i){
        swap(h.que[m], h.que[i]);
        min_heap(h, m);
    }
}

void push(heap &h, cost x)
{
    h.size++;
    h.que[h.size] = x;
    int i = h.size;
```

```

while (i > 1){
    if (h.que[i].w < h.que[i/2].w) swap(h.que[i], h.que[i/2]);
    else break;
    i = i/2;
}
}

```

```

cost top(heap &h)
{
    return h.que[1];
}

```

```

void pop(heap &h)
{
    if (isHeapEmpty(h)) return;
    h.que[1] = h.que[h.size];
    h.size--;
    if (h.size) min_heap(h, 1);
}

```

```

adjlist adj[N];
float a[N][N];

```

```

float DIJKSTRA[N];
int DIJKSTRA_TRACE[N];
float FLOYD[N][N];
int FLOYD_TRACE[N][N];

```

```

// =====
// =====THUẬT TOÁN DIJKSTRA=====
// =====

```

```

bool dijkstra(int s, adjlist a[], int n) {
    // Kiểm tra xem có cạnh âm hay không
    for (int i=1; i<=n; i++){
        for (int j=0; j<a[i].size; j++)
            if (a[i].e[j].w < 0.0f) return 0;
    }
    heap q(numEdges);
    bool kt[N];
    // Khởi tạo giá trị ban đầu
    for (int i=1; i<=n; i++){ DIJKSTRA[i] = inf; kt[i] = 0; }
    DIJKSTRA[s] = 0; DIJKSTRA_TRACE[s] = s;
}

```

```
push(q, (cost){s,0});
int u, i;
float w;

while(!isHeapEmpty(q)){
    u = top(q).v; // Tìm đỉnh cần dán nhãn
    pop(q);
    if (kt[u]) continue;
    kt[u] = 1; // Dán nhãn
    // Giãn cạnh
    for(int j=0; j<a[u].size; j++) {
        i = a[u].e[j].v;
        w = a[u].e[j].w;
        if(!kt[i] && DIJKSTRA[i] > DIJKSTRA[u] + w){
            DIJKSTRA[i] = DIJKSTRA[u] + w;
            DIJKSTRA_TRACE[i] = u;
            push(q, (cost){i, DIJKSTRA[i]});
        }
    }
}
return 1;
}
```

```
void get_path_dijkstra(int s, int t, int path[], int &n) {
    n = 0;
    int u = t;
    stack<int> d;
    d.push(t);
    while(u != s){ u = DIJKSTRA_TRACE[u]; d.push(u); }
    while(!d.empty()){
        path[n++] = d.top();
        d.pop();
    }
}
```

```
// =====
// =====THUẬT TOÁN FLOYD-WARSHALL=====
// =====
```

```
bool floyd(float a[][N], int n) {
    for (int i=1; i<=n; i++) for (int j=1; j<=n; j++) FLOYD[i][j] = a[i][j];
    for (int i=1; i<=n; i++) for (int j=1; j<=n; j++) FLOYD_TRACE[i][j] = j;
```

```
for (int k=1; k<=n; k++)
    for (int i=1; i<=n; i++)
        for (int j=1; j<=n; j++)
            if (FLOYD[i][k] + FLOYD[k][j] < FLOYD[i][j]){
                FLOYD[i][j] = FLOYD[i][k] + FLOYD[k][j];
                FLOYD_TRACE[i][j] = FLOYD_TRACE[i][k];
            }

// kiểm tra xem có chu trình âm hay không
for (int i=1; i<=n; i++)
    if (FLOYD[i][i] < 0.0f) return 0;
return 1;
}

void get_path_floyd(int s, int t, int path[], int &n) {
    n = 0;
    int u = s;
    path[n++] = s;
    while (u != t) {
        u = FLOYD_TRACE[u][t];
        path[n++] = u;
    }
}
```

physics.h

```
#include "header.h"

#define FORCE_CONSTANT 0.5f
#define FRICTION 0.2f
#define IDEAL_LENGTH 200.0f * ratio
#define AVOID_DISTANCE 50.0f * ratio
#define BEND_STRENGTH 100.0f * ratio
#define MAX_SPEED 100.0f

float GetDist(Vector2 p1, Vector2 p2)
{
    return sqrtf(powf(p2.x - p1.x, 2) + powf(p2.y - p1.y, 2));
}
```

```
float GetDotProduct(Vector2 v1, Vector2 v2)
{
    return v1.x * v2.x + v1.y * v2.y;
}
```

```
Vector2 GetMidPoint(Vector2 p1, Vector2 p2)
{
    return (Vector2){(p1.x + p2.x) / 2, (p1.y + p2.y) / 2};
}
```

```
Vector2 Vertex_VerexCompulsion(Vector2 p1, Vector2 p2)
{
    float d = GetDist(p1, p2);
    if (d >= IDEAL_LENGTH) return (Vector2){0,0};
    if (d < 1.0f) d = 1.0f;
    float f = FORCE_CONSTANT * (IDEAL_LENGTH - d);
    return (Vector2){(p1.x - p2.x) / d * f, (p1.y - p2.y) / d * f};
}
```

```
Vector2 AttractiveForce(Vector2 p1, Vector2 p2)
{
    float d = GetDist(p1, p2);
    if (d <= IDEAL_LENGTH) return (Vector2){0,0};
    float f = FORCE_CONSTANT * (d - IDEAL_LENGTH);
    if (d < 1.0f) d = 1.0f;
    return (Vector2){(p2.x - p1.x) / d * f, (p2.y - p1.y) / d * f};
}
```

```
void Vertex_EdgeCompulsion(Edge &edge, Node node)
{
    if (node.id == edge.src || node.id == edge.dest) return;
    float d, t, l;
    Vector2 n, a, b;

    a = {nodes[edge.dest].position.x - nodes[edge.src].position.x, nodes[edge.dest].position.y -
nodes[edge.src].position.y};
    l = GetDist(a, {0,0});
    b = {nodes[edge.src].position.x - node.position.x, nodes[edge.src].position.y -
node.position.y};
    t = - GetDotProduct(a, b) / (l*l);

    if (t < -1e-9 || t > 1.0f) return;
```

```
n = {b.x + a.x * t, b.y + a.y * t};

d = GetDist(n, {0,0});
if (d > AVOID_DISTANCE) return;
if (d < 1.0){
    n = {-a.y, a.x};
    d = GetDist(n, {0,0});
}
n = {n.x / d, n.y / d};
edge.mid.x += n.x * BEND_STRENGTH;
edge.mid.y += n.y * BEND_STRENGTH;
}

void Edge_EdgeCompulsion(Edge &edge1, Edge edge2)
{
    if (edge1.src != edge2.dest || edge1.dest != edge2.src) return;

    float d;
    Vector2 n;
    n = {nodes[edge1.src].position.y - nodes[edge1.dest].position.y,
nodes[edge1.dest].position.x - nodes[edge1.src].position.x};
    d = GetDist(n, {0,0});
    n = {n.x / d, n.y / d};

    edge1.mid.x += n.x * BEND_STRENGTH * 0.3;
    edge1.mid.y += n.y * BEND_STRENGTH * 0.3;
}

void KeepInScreen(Vector2 &p)
{
    if (p.x > centerX + centerW - GRAPH_PANEL_MARGIN) p.x = centerX + centerW -
GRAPH_PANEL_MARGIN;
    if (p.x < centerX + GRAPH_PANEL_MARGIN) p.x = centerX +
GRAPH_PANEL_MARGIN;
    if (p.y > panelY + panelH - GRAPH_PANEL_MARGIN) p.y = panelY + panelH -
GRAPH_PANEL_MARGIN;
    if (p.y < panelY + GRAPH_PANEL_MARGIN) p.y = panelY +
GRAPH_PANEL_MARGIN;
}

void PhysicsCalc()
{
    Vector2 DeltaVeloc;
```

```
float speed;

for (int i=1; i<=numNodes; i++) {
    if (nodes[i].isDragging) {
        nodes[i].position = mousePos;
        nodes[i].veloc = (Vector2){0,0};
        KeepInScreen(nodes[i].position);
        continue;
    }

    if (nodes[i].isFixed) continue;

    for (int j=1; j<=numNodes; j++) {
        if (i == j) continue;
        DeltaVeloc = Vertex_VerxCompulsion(nodes[i].position, nodes[j].position);
        nodes[i].veloc.x += DeltaVeloc.x;
        nodes[i].veloc.y += DeltaVeloc.y;
    }

    for (int j=0; j<numEdges; j++) {
        int other = -1;
        if (edges[j].src == i) other = edges[j].dest;
        else if (edges[j].dest == i) other = edges[j].src;
        if (other == -1) continue;
        DeltaVeloc = AttractiveForce(nodes[i].position, nodes[other].position);
        nodes[i].veloc.x += DeltaVeloc.x;
        nodes[i].veloc.y += DeltaVeloc.y;
    }

    speed = GetDist(nodes[i].veloc, {0,0});
    if (speed > MAX_SPEED){
        (nodes[i].veloc.x /= speed) *= MAX_SPEED;
        (nodes[i].veloc.y /= speed) *= MAX_SPEED;
    }

    nodes[i].position.x += nodes[i].veloc.x;
    nodes[i].position.y += nodes[i].veloc.y;
    nodes[i].veloc.x *= FRICTION;
    nodes[i].veloc.y *= FRICTION;

    KeepInScreen(nodes[i].position);
}
```

```
for (int i=0; i<numEdges; i++){
    edges[i].mid = GetMidPoint(nodes[edges[i].src].position, nodes[edges[i].dest].position);

    for (int j=1; j<=numNodes; j++)
        Vertex_EdgeCompulsion(edges[i], nodes[j]);

    for (int j=0; j<numEdges; j++)
        Edge_EdgeCompulsion(edges[i], edges[j]);
}
}
```

graphics.h

```
#include "header.h"
#include "algo.h"
#include "physics.h"
```

```
void DrawVertex(Node node)
{
    Color c;
    if (node.id == startNode) c = YELLOW;
    else if (node.id == endNode) c = BLUE;
    else c = WHITE;
    if (node.isDragging) c = ColorLerp(c, GRAY, 0.7);

    DrawCircleV(node.position, NODE_RADIUS, c);

    if (node.isFixed) DrawRing(node.position, NODE_RADIUS - 3, NODE_RADIUS, 0, 360,
36, BLACK);
    else DrawCircleLinesV(node.position, NODE_RADIUS, BLACK);

    float fontSize = 22 * stdratio;
    Vector2 textSize = MeasureTextEx(grpFont, TextFormat("%d", node.id), fontSize, 1);
    DrawTextEx(grpFont, TextFormat("%d", node.id), {node.position.x - textSize.x/2.0f,
node.position.y - textSize.y/2.0f}, fontSize, 1, BLACK);
}

void DrawEdge(Edge edge)
{
    Vector2 start = nodes[edge.src].position;
    Vector2 control = edge.mid;
```

```

Vector2 end = nodes[edge.dest].position;

Color c = edge.isPath ? RED : GRAY;
float border = edge.isPath ? LINE_WEIGHT*1.5 : LINE_WEIGHT;
if (isDirectedMode){
    Vector2 a = {control.x - end.x, control.y - end.y};

    float l = GetDist(a, (Vector2){0,0});
    float t = NODE_RADIUS / l;
    end.x += a.x * t;
    end.y += a.y * t;

    a = (Vector2){a.x/l, a.y/l};

    Vector2 n = {-a.y, a.x};
    DrawLineEx(end, (Vector2){end.x + a.x * 15 * ratio + n.x * 5 * ratio, end.y + a.y * 15 *
ratio + n.y * 5 * ratio}, border, c);
    DrawLineEx(end, (Vector2){end.x + a.x * 15 * ratio - n.x * 5 * ratio, end.y + a.y * 15 *
ratio - n.y * 5 * ratio}, border, c);
}

DrawSplineSegmentBezierQuadratic(start, control, end, border, c);

Vector2 midPoint;
midPoint.x = 0.25*start.x + 0.5*control.x + 0.25*end.x;
midPoint.y = 0.25*start.y + 0.5*control.y + 0.25*end.y;

float fontSize = 20 * stdratio;
Vector2 textSize = MeasureTextEx(stdFont, TextFormat("%.1f", edge.weight), fontSize,
1);
float padding = 3 * stdratio;

Rectangle bgRec = {
    midPoint.x - (textSize.x / 2.0f) - padding,
    midPoint.y - (textSize.y / 2.0f) - padding,
    textSize.x + padding*2,
    textSize.y + padding*2
};

DrawRectangleRec(bgRec, RAYWHITE);
DrawTextEx(stdFont, TextFormat("%.1f", edge.weight), {bgRec.x + padding, bgRec.y +
padding}, fontSize, 1, BLACK);
}

```

```
void ShuffleVertex()
{
    for (int i=1; i<=numNodes; i++){
        nodes[i].position = (Vector2){(float)GetRandomValue(centerX + 50, centerX + centerW
- 50), (float)GetRandomValue(panelY + 50, panelY + panelH - 50)};
    }
}

void EditGraphData(int visibleRows, int visibleCols)
{
    int len;
    if (IsKeyPressed(KEY_UP) && ptrY > 0) {
        if (ptrY == textScrollY) textScrollY--;
        ptrY--;
        len = (int)strlen(graphDataInput[ptrY]);
        if (ptrX > len) ptrX = len;
        if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
        if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
    }
    if (IsKeyPressed(KEY_DOWN) && ptrY < numRows - 1) {
        if (ptrY == textScrollY + visibleRows - 1) textScrollY++;
        ptrY++;
        len = (int)strlen(graphDataInput[ptrY]);
        if (ptrX >= len) ptrX = len;
        if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
        if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
    }
    if (IsKeyPressed(KEY_RIGHT)) {
        if (ptrX == (int)strlen(graphDataInput[ptrY]) && ptrY < numRows - 1){
            if (ptrY == textScrollY + visibleRows - 1) textScrollY++;
            ptrY++;
            ptrX = 0;
        }
        else ptrX++;
        if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
        if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
    }
}
```

```
}
if (IsKeyPressed(KEY_LEFT)) {
    if (ptrX == 0 && ptrY > 0){
        if (ptrY == textScrollY) textScrollY--;
        ptrY--;
        ptrX = (int)strlen(graphDataInput[ptrY]);
    }
    else if (ptrX > 0){
        ptrX--;
    }
    if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
    if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
}

if (IsKeyPressed(KEY_BACKSPACE)) {
    len = (int)strlen(graphDataInput[ptrY]);
    if (len > 0 && ptrX > 0) {
        for (int j=ptrX-1; j<len; j++) graphDataInput[ptrY][j] = graphDataInput[ptrY][j+1];
        if (ptrX == textScrollX) textScrollX--;
        ptrX--;
    } else if (len == 0 && ptrY > 0) {
        for (int j=ptrY; j<numRow-1; j++) strcpy(graphDataInput[j], graphDataInput[j+1]);
        numRows--;
        if (textScrollY > 0) textScrollY--;
        ptrY--;
        ptrX = (int)strlen(graphDataInput[ptrY]);
    }
    if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
    if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
}

if (IsKeyPressed(KEY_DELETE)) {
    len = (int)strlen(graphDataInput[ptrY]);
    if (ptrX < len && len > 0) {
        for (int j=ptrX; j<len; j++) graphDataInput[ptrY][j] = graphDataInput[ptrY][j+1];
    }
    if (ptrX == len && ptrY < numRows-1 && (int)strlen(graphDataInput[ptrY+1]) == 0){
        for (int j=ptrY+1; j<numRow-1; j++) strcpy(graphDataInput[j], graphDataInput[j+1]);
        numRows--;
    }
}
```

```
    }
    if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
    if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
}

if (IsKeyPressed(KEY_ENTER)) {
    if (ptrY == numRow - 1 && numRow < MAX_ROW) numRow++;
    if (ptrY == textScrollY + visibleRows - 1) textScrollY++;
    ptrY++;
    ptrX = 0;
    if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
    if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
}

len = (int)strlen(graphDataInput[ptrY]);
char key = GetCharPressed();
if (((key >= '0') && (key <= '9')) || (key == ' ') || (key == '.') || (key == '-') || (key == 'e'))
&& (len < MAX_COL - 1)) {
    for (int j=len; j>ptrX; j--) graphDataInput[ptrY][j] = graphDataInput[ptrY][j-1];
    if (ptrX == textScrollX + visibleCols) textScrollX++;
    graphDataInput[ptrY][ptrX++] = key;
    graphDataInput[ptrY][len + 1] = '\0';
    if (ptrX > textScrollX + visibleCols - 1 || ptrX < textScrollX) textScrollX = ptrX -
min(ptrX, visibleCols - 1);
    if (ptrY > textScrollY + visibleRows || ptrY < textScrollY) textScrollY = min(ptrY,
numRow - visibleRows);
}
}

void LoadGraphFromString(const char textData[][MAX_COL]) {
    if (sscanf(textData[0], "%d", &numNodes) < 1) return;    if (numNodes > 100) return;
    for (int i=1; i<=numNodes; i++) {
        nodes[i].id = i;
        nodes[i].veloc = (Vector2){0,0};
        nodes[i].isDragging = false;
        nodes[i].isFixed = false;
        KeepInScreen(nodes[i].position);
    }    for (int i=1; i<=numNodes; i++) {
        for (int j=1; j<=numNodes; j++) {
```

```
        a[i][j] = (i == j ? 0 : inf);
    }
}
free_adjlist(adj);

int row = 1;
int x, y;
float z; while (row <= numRows) {
    if (sscanf(textData[row++], "%d %d %f", &x, &y, &z) < 2) continue;

    if (z < a[x][y]) a[x][y] = z;
    else continue;
    push(adj[x], (cost){y, z});

    if (!isDirectedMode) {
        a[y][x] = a[x][y];
        push(adj[y], (cost){x, z});
    }
}

numEdges = 0;
for (int i=1; i<=numNodes; i++){
    for (int j=1; j<=numNodes; j++) if (a[i][j] < inf) {
        if (!isDirectedMode && i > j) continue;

        edges[numEdges].src = i;
        edges[numEdges].dest = j;
        edges[numEdges].weight = a[i][j];
        edges[numEdges].isPath = false;
        numEdges++;
    }
}

graphUpdated = true;
}

void LoadStringFromFile(char *filePath){
    FILE *fp = fopen(filePath, "r");

    if (fp != NULL) {
        for (int i=0; i<numRow; i++) strcpy(graphDataInput[i], "");
        int rowptr = 0;
        while (numRow < MAX_ROW && fgets(graphDataInput[rowptr], MAX_COL, fp)) {
```

```
graphDataInput[rowptr][strcspn(graphDataInput[rowptr], "\r\n")] = '\0';
if (rowptr == numRows - 1) numRows++;
rowptr++;
}

sprintf(statusMsg, "Đã tải file: %s", fileNameInput);
fclose(fp);
} else {
    sprintf(statusMsg, "LỖI: Không tìm thấy file!");
}
}

void SaveFileFromString(char *filePath){
    FILE *fp = fopen(filePath, "w");

    if (fp != NULL) {
        fprintf(fp, graphOutput);

        sprintf(statusMsg, "Đã lưu file: %s", fileNameOutput);
        fclose(fp);
    } else {
        sprintf(statusMsg, "LỖI: Không thể lưu file!");
    }
}

void HighlightPath(int pathNodes[], int n) {
    for (int i=0; i<numEdges; i++) edges[i].isPath = false;

    for (int i=0; i<n-1; i++) {
        int u = pathNodes[i];
        int v = pathNodes[i+1];
        for (int j=0; j<numEdges; j++) {
            if (isDirectedMode) {
                if (edges[j].src == u && edges[j].dest == v) edges[j].isPath = true;
            } else {
                if ((edges[j].src == u && edges[j].dest == v) || (edges[j].src == v && edges[j].dest
== u)) edges[j].isPath = true;
            }
        }
    }
}

void FindShortestPath()
```

```
{
    int path[N], pathLength = 0;

    char *startByte, *currByte = graphOutput;

    if (graphUpdated) {
        isFloydValid = floyd(a, numNodes);

        isDijkstraValid = dijkstra(startNode, adj, numNodes);
        preStartNode = startNode;

        graphUpdated = false;
    }

    if (algoChoice == 0) {
        if (preStartNode != startNode){
            isDijkstraValid = dijkstra(startNode, adj, numNodes);
            preStartNode = startNode;
        }

        if (isDijkstraValid){
            if (DIJKSTRA[endNode] < inf){
                get_path_dijkstra(startNode, endNode, path, pathLength);

                sprintf(statusMsg, "Đã tìm ra đường đi ngắn nhất");

                startByte = currByte += sprintf(currByte, "Dijkstra: Chi phí = %.2fn",
                DIJKSTRA[endNode]);
                currByte += sprintf(currByte, "Đường đi: %d ", path[0]);
                for (int i=1; i<pathLength; i++){
                    if ((currByte - startByte) >= 22){
                        currByte += sprintf(currByte, "\n");
                        startByte = currByte;
                    }
                    currByte += sprintf(currByte, "-> %d ", path[i]);
                }
            } else {
                sprintf(statusMsg, "Không tìm thấy đường đi");
                strcpy(graphOutput, "");
            }
        } else {
            sprintf(statusMsg, "LỖI: Tồn tại cạnh âm!");
            strcpy(graphOutput, "");
        }
    }
```

```
    }
} else {
    if (isFloydValid){
        if (FLOYD[startNode][endNode] < inf){
            get_path_floyd(startNode, endNode, path, pathLength);

            sprintf(statusMsg, "Đã tìm ra đường đi ngắn nhất");

            startByte = currByte += sprintf(currByte, "Floyd: Chi phí = %.2f\n",
FLOYD[startNode][endNode]);
            currByte += sprintf(currByte, "Đường đi: %d ", path[0]);
            for (int i=1; i<pathLength; i++){
                if ((currByte - startByte) >= 22){
                    currByte += sprintf(currByte, "\n");
                    startByte = currByte;
                }
                currByte += sprintf(currByte, "-> %d ", path[i]);
            }
        } else {
            sprintf(statusMsg, "Không tìm thấy đường đi");
            strcpy(graphOutput, "");
        }
    } else {
        sprintf(statusMsg, "LỖI: Tồn tại chu trình âm!");
        strcpy(graphOutput, "");
    }
}

if (pathLength) HighlightPath(path, pathLength);
}

void UpdateLayout()
{
    padding = sw * 0.015f;
    topH = sh * 0.06f;
    leftW = rightW = sw * 0.25f;
    centerW = sw - leftW - rightW - (padding * 4);
    leftX = padding;
    centerX = leftX + leftW + padding;
    rightX = centerX + centerW + padding;
    panelY = topH + padding * 2;
    panelH = sh - topH - (padding * 3);
}
```

```
bigFontsz = min(sw, sh) * 0.028;
}

void SetUpUI()
{
    ClearBackground(RAYWHITE);

    // Tên đề tài
    DrawTextEx(stdFont, "PBL1 - Lập trình đồ án tính toán\nĐề tài: Tìm đường đi ngắn nhất
(Floyd, Dijkstra)\nGiảng viên hướng dẫn: Đỗ Thị Tuyết Hoa", (Vector2){leftX,
padding*0.5f}, bigFontsz, 1.0f, DARKBLUE);
    DrawTextEx(stdFont, "Sinh viên thực hiện:\n-Nguyễn Công Dũng - 25T_DT3\n-Nguyễn
Văn Tài - 25T_DT3", (Vector2){leftX + 0.36f*sw, padding*0.5f}, bigFontsz, 1.0f,
DARKBLUE);

    // ----- TOP MENU -----
    float menuW = fmin(200.0f, sw * 0.33f);
    GuiToggleGroup((Rectangle){ sw - menuW*2 - padding, padding, menuW, topH }, "Vô
hướng;Có hướng", &dum1);
    if (dum1 != prevDum1) {
        isDirectedMode = (dum1 == 1);
        LoadGraphFromString(graphDataInput);
        prevDum1 = dum1;
    }

    // ----- LEFT PANEL -----
    GuiPanel((Rectangle){ leftX, panelY, leftW, panelH }, "Quản lý dữ liệu");

    float innerPad = leftW * 0.08f;
    float innerW = leftW - (innerPad * 2);
    float currentY = panelY + 35.0f;

    // Nhập tên file input
    DrawTextEx(stdFont, "1. Dữ liệu từ File:", (Vector2){leftX + innerPad, currentY},
bigFontsz, 1, DARKBLUE);
    currentY += 25.0f;

    GuiSetStyle(TEXTBOX, TEXT_COLOR_PRESSED, ColorToInt(DARKBLUE));
    if (GuiTextBox((Rectangle){ leftX + innerPad, currentY, innerW, 35.0f }, fileNameInput,
256, editInputFileNameMode)) {
        editInputFileNameMode = !editInputFileNameMode;
        editGraphDataMode = editOutputFileNameMode = editEnd = editStart = false;
    }
}
```

```
currentY += 45.0f;

// Nút cập nhật dữ liệu từ file
float buttonH = 40.0f;
if (GuiButton((Rectangle){ leftX + innerPad, currentY, innerW, buttonH }, "#01# TẢI
FILE")) {
    char filePathInput[512];
    sprintf(filePathInput, "test/%s", fileNameInput);

    LoadStringFromFile(filePathInput);
    LoadGraphFromString(graphDataInput);

    editInputFileNameMode = false;
}
currentY += 60.0f;

// Hiện thị và thay đổi data
DrawTextEx(stdFont, "2. Chính sửa thủ công:", (Vector2){leftX + innerPad, currentY},
bigFontSize, 1, DARKBLUE);
currentY += 25.0f;

float gutterWidth = 40.0f;
float textH = (panelY + panelH) - currentY - buttonH - 15.0f;
float textX = leftX + innerPad + gutterWidth;
float rowMargin = 2;
float textSize = 16;
float textPad = 5;
Rectangle textArea = { leftX + innerPad + gutterWidth, currentY, innerW - gutterWidth,
textH };
Vector2 charW = MeasureTextEx(dataFont, "a", textSize, 1.0f);
float lineHeight = charW.y;
int visibleRows = (int)((textH - textPad) / (lineHeight + rowMargin));
int visibleCols = (int)((innerW - gutterWidth - textPad) / (charW.x + 1.0f));

DrawRectangle(leftX + innerPad, currentY, gutterWidth, textH, LIGHTGRAY);
DrawRectangleLinesEx(textArea, 1, LIGHTGRAY);

if (CheckCollisionPointRec(mousePos, textArea)){
    if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)){
        editGraphDataMode = true;

        ptrX = textScrollX + (int) (mousePos.x - textX) / (charW.x + 1.0f);
```

```
ptrY = textScrollY + (int) (mousePos.y - currentY - textPad) / (charW.y +
rowMargin);
ptrX = min(ptrX, (int)strlen(graphDataInput[ptrY]));
}

if (wheelMove != 0.0f){
    if (wheelMove > 0.0f && textScrollY > 0) textScrollY--;
    else if (wheelMove < 0.0f && textScrollY + visibleRows <= numRows)
textScrollY++;
}
}

if (editGraphDataMode) EditGraphData(visibleRows, visibleCols);

char realText[visibleCols + 1];
int endRow = min(textScrollY + visibleRows, numRows);
Rectangle lineRect = { textX, currentY + textPad, leftW - 2*innerPad - gutterWidth,
lineHeight };

for (int i = textScrollY; i < endRow; i++) {
    if (i == ptrY && editGraphDataMode) {
        DrawRectangleRec(lineRect, Fade(SKYBLUE, 0.3f));

        if (((int)(GetTime() * 2) % 2 == 0)) {
            float ptrPos = 5 + (ptrX - textScrollX) * (charW.x + 1.0f) - 1.0f;
            DrawRectangle(lineRect.x + ptrPos, lineRect.y, 2, lineHeight, BLACK);
        }
    }

    for (int j = textScrollX; j < textScrollX + visibleCols; j++){
        if ((realText[j - textScrollX] = graphDataInput[i][j]) == '\0') break;
        realText[j+1] = '\0';
    }

    DrawTextEx(dataFont, TextFormat("%d", i), {leftX + innerPad + 10, lineRect.y},
textSize, 1.0f, DARKGRAY);
    DrawTextEx(dataFont, realText, {lineRect.x + textPad, lineRect.y}, textSize, 1.0f,
BLACK);

    lineRect.y += rowMargin + lineHeight;
}

// Nút cập nhật đồ thị
```

```
if (GuiButton((Rectangle){ leftX + innerPad, currentY + textH + 5.0f, innerW, buttonH },
"CẬP NHẬT ĐỒ THỊ")) {
    LoadGraphFromString(graphDataInput);
    sprintf(statusMsg, "Đồ thị đã được cập nhật!");

    editGraphDataMode = false;
}

// ----- GRAPH PANEL -----
Rectangle graphArea = {centerX, panelY, centerW, panelH};
DrawRectangleLinesEx(graphArea, 2.0f, GRAY);

if (CheckCollisionPointRec(mousePos, graphArea) && wheelMove != 0.0f){
    if (wheelMove < 0.0f && ratio > 0.2f) ratio -= 0.02;
    else if (wheelMove > 0.0f && ratio < 1.0f) ratio += 0.02;
    stdratio = fmax(ratio, 0.6f);
}

// ----- RIGHT PANEL -----
GuiPanel((Rectangle){ rightX, panelY, rightW, panelH }, "Thanh công cụ");

float rPad = rightW * 0.08f;
float rInW = rightW - (rPad * 2);

float rY = panelY + 40.0f;
float rowH = panelH * 0.07f;
float spacing = panelH * 0.01f;

// Chọn thuật toán
GuiLabel((Rectangle){ rightX + rPad, rY, rInW * 0.4f, rowH }, "Thuật toán");
GuiComboBox((Rectangle){ rightX + rPad + (rInW * 0.4f), rY, rInW * 0.6f, rowH },
"Dijkstra;Floyd", &algoChoice);

rY += rowH + spacing;

// Nhập đỉnh bắt đầu
GuiLabel((Rectangle){ rightX + rPad, rY, rInW * 0.4f, rowH }, "Bắt đầu:");
if (GuiValueBox((Rectangle){ rightX + rPad + (rInW * 0.4f), rY, rInW * 0.6f, rowH },
NULL, &startNode, 1, numNodes, editStart)) {
    editStart = !editStart;
    editInputFileNameMode = editOutputFileNameMode = editGraphDataMode = false;
}
rY += rowH + spacing;
```

```
// Nhập đỉnh kết thúc
GuiLabel((Rectangle){ rightX + rPad, rY, rInW * 0.4f, rowH }, "Kết thúc:");
if (GuiValueBox((Rectangle){ rightX + rPad + (rInW * 0.4f), rY, rInW * 0.6f, rowH },
NULL, &endNode, 1, numNodes, editEnd)) {
    editEnd = !editEnd;
    editInputFileNameMode = editOutputFileNameMode = editGraphDataMode = false;
}
rY += rowH + spacing * 1.5f;

// Nút thực thi thuật toán tìm đường đi ngắn nhất
if (GuiButton((Rectangle){ rightX + rPad, rY, rInW, rowH }, "TÌM ĐƯỜNG ĐI")) {
    FindShortestPath();
}
rY += rowH + spacing * 1.5f;

// Nút xáo trộn đỉnh
if (GuiButton((Rectangle){ rightX + rPad, rY, rInW, rowH }, "XÁO ĐỈNH"))
ShuffleVertex();
rY += rowH + spacing * 1.5f;

// Hiện thị trạng thái
DrawTextEx(stdFont, "Trạng thái:", {rightX + rPad, rY}, bigFontsz, 1, GRAY);
DrawTextEx(stdFont, statusMsg, {rightX + rPad, rY + panelH * 0.04f}, bigFontsz, 1,
RED);
rY += rowH + spacing;

// Hiện thị kết quả
float outTextH = panelH * 0.24f;
GuiTextBox((Rectangle){rightX + rPad, rY, rInW, outTextH}, graphOutput, bigFontsz,
false);
rY += outTextH + spacing;

// Nhập file output
GuiLabel((Rectangle){ rightX + rPad, rY, rInW * 0.3f, rowH }, "File out:");
GuiSetStyle(TEXTBOX, TEXT_COLOR_PRESSED, ColorToInt(DARKBLUE));
if (GuiTextBox((Rectangle){ rightX + rPad + rInW * 0.35f, rY, rInW * 0.65f, rowH },
fileNameOutput, 256, editOutputFileNameMode)) {
    editOutputFileNameMode = !editOutputFileNameMode;
    editGraphDataMode = editInputFileNameMode = false;
}
rY += rowH + spacing;
```

```
// Lưu kết quả file
if (GuiButton((Rectangle){ rightX + rPad, rY, rInW, rowH }, "#02# LƯU FILE")) {
    char filePathOutput[512];
    sprintf(filePathOutput, "test/%s", fileNameOutput);

    SaveFileFromString(filePathOutput);

    editOutputFileNameMode = false;
}
}

void MouseCheck(Vector2 mousePos)
{
    float deltaTime = GetFrameTime();
    if (doubleClickTimer > 0) doubleClickTimer -= deltaTime;
    else clickCount = 0;

    if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
        for (int i=1; i<=numNodes; i++) {
            if (CheckCollisionPointCircle(mousePos, nodes[i].position, NODE_RADIUS)) {
                clickCount++;
                if (clickCount == 1) {
                    doubleClickTimer = 0.25f;
                    selectedNodeIndex = i;
                    nodes[i].isDragging = true;
                }
                else if (clickCount == 2 && selectedNodeIndex == i) {
                    nodes[i].isFixed = !nodes[i].isFixed;
                    clickCount = 0;
                    doubleClickTimer = 0;
                    nodes[i].isDragging = false;
                }
                break;
            }
        }
    }
    if (IsMouseButtonReleased(MOUSE_LEFT_BUTTON)) {
        for (int i=1; i<=numNodes; i++)
            nodes[i].isDragging = false;
    }
}
```

```
void DrawGraph()
{
    MouseCheck(mousePos);

    PhysicsCalc();

    for (int i=0; i<numEdges; i++) DrawEdge(edges[i]);
    for (int i=1; i<=numNodes; i++) DrawVertex(nodes[i]);
}
```

main.cpp

```
#include "header.h"
#include "graphics.h"
using namespace std;

int main() {
    SetConfigFlags(FLAG_WINDOW_RESIZABLE | FLAG_WINDOW_HIGHDPI);
    InitWindow(1200, 700, "Tim duong di ngan nhât");
    SetTargetFPS(60);

    stdFont = LoadFontEx("resources/fonts/static/Roboto_SemiCondensed-Medium.ttf", 50,
codepoints, codepointsCount);
    grpFont = LoadFontEx("resources/fonts/static/Roboto_Condensed-Black.ttf", 50,
codepoints, codepointsCount);
    dataFont = LoadFontEx("resources/fonts/consolas/Consolas-Regular.ttf", 50, codepoints,
codepointsCount);
    GuiSetFont(stdFont);
    GuiSetStyle(DEFAULT, TEXT_SIZE, 20);
    UnloadCodepoints(codepoints);

    dum1 = prevDum1 = isDirectedMode;

    LoadGraphFromString(graphDataInput);

    while (!WindowShouldClose()) {
        sw = GetScreenWidth();
        sh = GetScreenHeight();
        UpdateLayout();

        mousePos = GetMousePosition();
```

```
wheelMove = GetMouseWheelMove();

BeginDrawing();
SetUpUI();
DrawGraph();
EndDrawing();
}

CloseWindow();
return 0;
}
```